

Programming Language Syntax

EXAMPLE 2.1 Syntax of Arabic numerals

Unlike natural languages such as English or Chinese, computer languages must be precise. Both their form (syntax) and meaning (semantics) must be specified without ambiguity, so that both programmers and computers can tell what a program is supposed to do. To provide the needed degree of precision, language designers and implementors use formal syntactic and semantic notation. To facilitate the discussion of language features in later chapters, we will cover this notation first: syntax in the current chapter and semantics in [Chapter 4](#).

As a motivating example, consider the Arabic numerals with which we represent numbers. These numerals are composed of digits, which we can enumerate as follows (‘|’ means “or”):

$$\text{digit} \longrightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Digits are the syntactic building blocks for numbers. In the usual notation, we say that a natural number is represented by an arbitrary-length (nonempty) string of digits, beginning with a nonzero digit:

$$\begin{aligned}\text{non_zero_digit} &\longrightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \\ \text{natural_number} &\longrightarrow \text{non_zero_digit digit}^*\end{aligned}$$

Here the “Kleene¹ star” metasympol (*) is used to indicate zero or more repetitions of the symbol to its left. ■

Of course, digits are only symbols: ink blobs on paper or pixels on a screen. They carry no meaning in and of themselves. We add semantics to digits when we say that they represent the natural numbers from zero to nine, as defined by mathematicians. Alternatively, we could say that they represent colors, or the days of the week in a decimal calendar. These would constitute alternative semantics for the same syntax. In a similar fashion, we define the semantics of natural

¹ Stephen Kleene (1909–1994), a mathematician at the University of Wisconsin, was responsible for much of the early development of the theory of computation, including much of the material in Section 2.4.

numbers by associating a base-10, place-value interpretation with each string of digits. Similar syntax rules and semantic interpretations can be devised for rational numbers, (limited-precision) real numbers, arithmetic, assignments, control flow, declarations, and indeed all of programming languages.

Distinguishing between syntax and semantics is useful for at least two reasons. First, different programming languages often provide features with very similar semantics but very different syntax. It is generally much easier to learn a new language if one is able to identify the common (and presumably familiar) ideas beneath the unfamiliar syntax. Second, there are some very efficient and elegant algorithms that a compiler or interpreter can use to discover the syntactic structure (but not the semantics!) of a computer program, and these algorithms can be used to drive the rest of the compilation or interpretation process.

In the current chapter we focus on syntax: how we specify the structural rules of a programming language, and how a compiler identifies the structure of a given input program. These two tasks—specifying syntax rules and figuring out how (and whether) a given program was built according to those rules—are distinct. The first is of interest mainly to programmers, who want to write valid programs. The second is of interest mainly to compilers, which need to analyze those programs. The first task relies on *regular expressions* and *context-free grammars*, which specify how to generate valid programs. The second task relies on *scanners* and *parsers*, which recognize program structure. We address the first of these tasks in [Section 2.1](#), the second in [Sections 2.2](#) and [2.3](#).

In [Section 2.4](#) (largely on the PLP CD) we take a deeper look at the formal theory underlying scanning and parsing. In theoretical parlance, a scanner is a *deterministic finite automaton* (DFA) that recognizes the tokens of a programming language. A parser is a deterministic *push-down automaton* (PDA) that recognizes the language's context-free syntax. It turns out that one can generate scanners and parsers automatically from regular expressions and context-free grammars. This task is performed by tools like Unix's `lex` and `yacc`.² Possibly nowhere else in computer science is the connection between theory and practice so clear and so compelling.

2.1 Specifying Syntax: Regular Expressions and Context-Free Grammars

Formal specification of syntax requires a set of rules. How complicated (expressive) the syntax can be depends on the kinds of rules we are allowed to use.

² At many sites, `lex` and `yacc` have been superseded by the GNU `flex` and `bison` tools. These independently developed, noncommercial alternatives are available without charge from the Free Software Foundation at www.gnu.org/software. They provide a superset of the functionality of `lex` and `yacc`.

It turns out that what we intuitively think of as tokens can be constructed from individual characters using just three kinds of formal rules: concatenation, alternation (choice among a finite set of alternatives), and so-called “Kleene closure” (repetition an arbitrary number of times). Specifying most of the rest of what we intuitively think of as syntax requires one additional kind of rule: recursion (creation of a construct from simpler instances of the same construct). Any set of strings that can be defined in terms of the first three rules is called a *regular set*, or sometimes a *regular language*. Regular sets are generated by *regular expressions* and recognized by scanners. Any set of strings that can be defined if we add recursion is called a *context-free language* (CFL). Context-free languages are generated by *context-free grammars* (CFGs) and recognized by parsers. (Terminology can be confusing here. The meaning of the word “language” varies greatly, depending on whether we’re talking about “formal” languages [e.g., regular or context-free], or programming languages. A formal language is just a set of strings, with no accompanying semantics.)

2.1.1 Tokens and Regular Expressions

Tokens are the basic building blocks of programs—the shortest strings of characters with individual meaning. Tokens come in many *kinds*, including keywords, identifiers, symbols, and constants of various types. Some kinds of token (e.g., the increment operator) correspond to only one string of characters. Others (e.g., *identifier*) correspond to a set of strings that share some common form. (In most languages, keywords are special strings of characters that have the right form to be identifiers, but are reserved for special purposes.) We will use the word “token” informally to refer to both the generic kind (an identifier, the increment operator) and the specific string (`foo`, `++`); the distinction between these should be clear from context.

EXAMPLE 2.2

Lexical structure of C99

Some languages have only a few kinds of token, of fairly simple form. Other languages are more complex. C, for example, has almost 100 kinds of tokens, including 37 keywords (`double`, `if`, `return`, `struct`, etc.); identifiers (`my_variable`, `your_type`, `sizeof`, `printf`, etc.); integer (0765, 0x1f5, 501), floating-point (6.022e23), and character (`'x'`, `'\''`, `'\0170'`) constants; string literals (`"snerk"`, `"say \hi\n"`); 54 “punctuators” (`+`, `]`, `->`, `*=`, `:`, `||`, etc.), and two different forms of comments. There are provisions for international character sets, string literals that span multiple lines of source code, constants of varying precision (width), alternative “spellings” for symbols that are missing on certain input devices, and preprocessor macros that build tokens from smaller pieces. Other large, modern languages (Java, Ada 95) are similarly complex. ■

To specify tokens, we use the notation of *regular expressions*. A regular expression is one of the following.

1. A character
2. The empty string, denoted ϵ

3. Two regular expressions next to each other, meaning any string generated by the first one followed by (concatenated with) any string generated by the second one
4. Two regular expressions separated by a vertical bar (`|`), meaning any string generated by the first one *or* any string generated by the second one
5. A regular expression followed by a Kleene star, meaning the concatenation of zero or more strings generated by the expression in front of the star

Parentheses are used to avoid ambiguity about where the various subexpressions start and end.³

Consider, for example, the syntax of numeric constants accepted by a simple hand-held calculator:

EXAMPLE 2.3

Syntax of numeric constants

```

number  $\rightarrow$  integer | real
integer  $\rightarrow$  digit digit*
real  $\rightarrow$  integer exponent | decimal ( exponent |  $\epsilon$  )
decimal  $\rightarrow$  digit* ( . digit | digit . ) digit*
exponent  $\rightarrow$  ( e | E ) ( + | - |  $\epsilon$  ) integer
digit  $\rightarrow$  0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

The symbols to the left of the $\rightarrow$ signs provide names for the regular expressions. One of these (*number*) will serve as a token name; the others are simply for convenience in building larger expressions.⁴ Note that while we have allowed definitions to build on one another, nothing is ever defined in terms of itself, even indirectly. Such recursive definitions are the distinguishing characteristic of context-free grammars, described in the [Section 2.1.2](#). To generate a valid number, we expand out the sub-definitions and then scan the resulting expression from left to right, choosing among alternatives at each vertical bar, and choosing a number of repetitions at each Kleene star. Within each repetition we may make different choices at vertical bars, generating different substrings. ■

Character Sets and Formatting Issues

Upper- and lowercase letters in identifiers and keywords are considered distinct in some languages (e.g., Modula-2/3 and C and its descendants), and identical in others (e.g., Ada, Common Lisp, Fortran 90, and Pascal). Thus `foo`, `Foo`, and `F00` all represent the same identifier in Ada, but different identifiers in C. Modula-2 and Modula-3 require keywords and predefined (built-in) identifiers to be written

3 Some authors use λ to represent the empty string. Some use a period (`.`), rather than juxtaposition, to indicate concatenation. Some use a plus sign (`+`), rather than a vertical bar, to indicate alternation.

4 We have assumed here that all numeric constants are simply “numbers.” In many programming languages, integer and real constants are separate kinds of token. Their syntax may also be more complex than indicated here, to support such features as multiple lengths or nondecimal bases.

in uppercase; C and its descendants require them to be written in lowercase. A few languages (notably Modula-3 and Standard Pascal) allow only letters and digits in identifiers. Most (including many actual implementations of Pascal) allow underscores. A few (notably Lisp) allow a variety of additional characters. Some languages (e.g., Java, C#, and Modula-3) have standard conventions on the use of upper- and lowercase letters in names.⁵

With the globalization of computing, non-Latin character sets have become increasingly important. Many modern languages, including C99, C++, Ada 95, Java, C#, and Fortran 2003 have explicit support for multibyte character sets, generally based on the Unicode and ISO/IEC 10646 international standards. Most modern programming languages allow non-Latin characters to appear within comments and character strings; an increasing number allow them in identifiers as well. Conventions for portability across character sets and for *localization* to a given character set can be surprisingly complex, particularly when various forms of backward compatibility are required (the C99 Rationale devotes five full pages to this subject [Int99, pp. 19–23]); for the most part we ignore such issues here.

Some language implementations impose limits on the maximum length of identifiers, but most avoid such unnecessary restrictions. Most modern languages are also more-or-less *free format*, meaning that a program is simply a sequence of tokens: what matters is their order with respect to one another, not their physical position within a printed line or page. “White space” (blanks, tabs, carriage returns, and line and page feed characters) between tokens is usually ignored, except to the extent that it is needed to separate one token from the next. There are a few exceptions to these rules. Some language implementations limit the maximum length of a line, to allow the compiler to store the current line in a fixed-length buffer. Dialects of Fortran prior to Fortran 90 use a *fixed format*, with 72 characters per line (the width of a paper punch card, on which programs were once stored), and with different columns within the line reserved for different purposes. Linebreaks serve to separate statements in several other languages, including Haskell, Occam, SR, Tcl, and Python. Haskell, Occam, and Python also give special

DESIGN & IMPLEMENTATION

Formatting restrictions

Formatting limitations inspired by implementation concerns—as in the punch-card-oriented rules of Fortran 77 and its predecessors—have a tendency to become unwanted anachronisms as implementation techniques improve. Given the tendency of certain word processors to “fill” or auto-format text, the linebreak and indentation rules of languages like Haskell, Occam, and Python are somewhat controversial.

⁵ For the sake of consistency we do not always obey such conventions in this book: most examples follow the common practice of C programmers, in which underscores, rather than capital letters, separate the “subwords” of names.

significance to indentation. The body of a loop, for example, consists of precisely those subsequent lines that are indented farther than the header of the loop.

Other Uses of Regular Expressions

Many readers will be familiar with regular expressions from the `grep` family of tools in Unix, the search facilities of various text editors (notably `emacs`), or such scripting languages and tools as Perl, Python, Ruby, `awk`, and `sed`. Most of these provide a rich set of extensions to the notation of regular expressions. Some extensions, such as shorthand for “zero or one occurrences” or “anything other than white space,” do not change the power of the notation. Others, such as the ability to require a second occurrence, later in the input string, of the same character sequence that matched an earlier part of the expression, increase the power of the notation, so that it is no longer restricted to generating regular sets. Still other extensions are designed not to increase the expressiveness of the notation but rather to tie it to other language facilities. In many tools, for example, one can bracket portions of a regular expression in such a way that when a string is matched against it the contents of the corresponding substrings are assigned into named local variables. We will return to these issues in [Section 13.4.2](#), in the context of scripting languages.

2.1.2 Context-Free Grammars

EXAMPLE 2.4

Syntactic nesting in expressions

Regular expressions work well for defining tokens. They are unable, however, to specify *nested* constructs, which are central to programming languages. Consider for example the structure of an arithmetic expression.

$$\begin{aligned} \text{expr} &\longrightarrow \text{id} \mid \text{number} \mid - \text{expr} \mid (\text{expr}) \\ &\quad \mid \text{expr op expr} \\ \text{op} &\longrightarrow + \mid - \mid * \mid / \end{aligned}$$

Here the ability to define a construct in terms of itself is crucial. Among other things, it allows us to ensure that left and right parentheses are matched, something that cannot be accomplished with regular expressions (see [Section 2.4.3](#) for more details). The arrow symbol ($\longrightarrow$) means “can have the form”; for brevity it is sometimes pronounced “goes to.” ■

Each of the rules in a context-free grammar is known as a *production*. The symbols on the left-hand sides of the productions are known as *variables*, or *nonterminals*. There may be any number of productions with the same left-hand side. Symbols that are to make up the strings derived from the grammar are known as *terminals* (shown here in typewriter font). They cannot appear on the left-hand side of any production. In a programming language, the terminals of the context-free grammar are the language’s tokens. One of the nonterminals, usually the one on the left-hand side of the first production, is called the *start symbol*. It names the construct defined by the overall grammar.

The notation for context-free grammars is sometimes called Backus-Naur Form (BNF), in honor of John Backus and Peter Naur, who devised it for the definition of the Algol-60 programming language [NBB⁺63].⁶ Strictly speaking, the Kleene star and meta-level parentheses of regular expressions are not allowed in BNF, but they do not change the expressive power of the notation, and are commonly included for convenience. Sometimes one sees a “Kleene plus” (+) as well; it indicates one or more instances of the symbol or group of symbols in front of it.⁷ When augmented with these extra operators, the notation is often called extended BNF (EBNF). The construct

EXAMPLE 2.5
Extended BNF (EBNF)

$$id_list \longrightarrow id (, id)^*$$

is shorthand for

$$id_list \longrightarrow id$$

$$id_list \longrightarrow id_list , id$$

“Kleene plus” is analogous. Note that the parentheses here are metasymbols. In [Example 2.4](#) they were part of the language being defined, and were written in fixed-width font.⁸

Like the Kleene star and parentheses, the vertical bar is in some sense superfluous, though it was provided in the original BNF. The construct

$$op \longrightarrow + \mid - \mid * \mid /$$

can be considered shorthand for

$$op \longrightarrow +$$

$$op \longrightarrow -$$

$$op \longrightarrow *$$

$$op \longrightarrow /$$

which is also sometimes written

$$op \longrightarrow +$$

$$\longrightarrow -$$

$$\longrightarrow *$$

$$\longrightarrow /$$

6 John Backus (1924–2007) was also the inventor of Fortran. He spent most of his professional career at IBM Corporation, and was named an IBM Fellow in 1987. He received the ACM Turing Award in 1977.

7 Some authors use curly braces ({ }) to indicate zero or more instances of the symbols inside. Some use square brackets ([]) to indicate zero or one instances of the symbols inside—that is, to indicate that those symbols are optional. In both regular and extended BNF, many authors use ::= instead of $\longrightarrow$.

8 To avoid confusion, some authors place quote marks around any single character that is part of the language being defined: $id_list \longrightarrow id (' , ' id)^*$; $expr \longrightarrow (' expr ')$.

Many tokens, such as `id` and `number` above, have many possible spellings (i.e., may be represented by many possible strings of characters). The parser is oblivious to these; it does not distinguish one identifier from another. The semantic analyzer does distinguish them, however; the scanner must save the spelling of each such “interesting” token for later use.

2.1.3 Derivations and Parse Trees

A context-free grammar shows us how to generate a syntactically valid string of terminals: Begin with the start symbol. Choose a production with the start symbol on the left-hand side; replace the start symbol with the right-hand side of that production. Now choose a nonterminal A in the resulting string, choose a production P with A on its left-hand side, and replace A with the right-hand side of P . Repeat this process until no nonterminals remain.

As an example, we can use our grammar for expressions to generate the string “`slope * x + intercept`”:

EXAMPLE 2.6

Derivation of `slope * x + intercept`

$$\begin{aligned}
 \text{expr} &\Rightarrow \text{expr op expr} \\
 &\Rightarrow \text{expr op id} \\
 &\Rightarrow \underline{\text{expr}} + \text{id} \\
 &\Rightarrow \text{expr op } \underline{\text{expr}} + \text{id} \\
 &\Rightarrow \text{expr op id} + \text{id} \\
 &\Rightarrow \underline{\text{expr}} * \text{id} + \text{id} \\
 &\Rightarrow \text{id} * \text{id} + \text{id} \\
 &\quad (\text{slope}) \quad (\text{x}) \quad (\text{intercept})
 \end{aligned}$$

The $\Rightarrow$ metasymbol is often pronounced “derives.” It indicates that the right-hand side was obtained by using a production to replace some nonterminal in the left-hand side. At each line we have underlined the symbol A that is replaced in the following line. ■

A series of replacement operations that shows how to derive a string of terminals from the start symbol is called a *derivation*. Each string of symbols along the way is called a *sentential form*. The final sentential form, consisting of only terminals, is called the *yield* of the derivation. We sometimes elide the intermediate steps and write $\text{expr} \Rightarrow^* \text{slope} * \text{x} + \text{intercept}$, where the metasymbol $\Rightarrow^*$ means “derives after zero or more replacements.” In this particular derivation, we have chosen at each step to replace the right-most nonterminal with the right-hand side of some production. This replacement strategy leads to a *right-most* derivation. There are many other possible derivations, including *left-most* and options in-between.

We saw in [Chapter 1](#) that we can represent a derivation graphically as a *parse tree*. The root of the parse tree is the start symbol of the grammar. The leaves of the tree are its yield. Each internal node, together with its children, represents the use of a production.

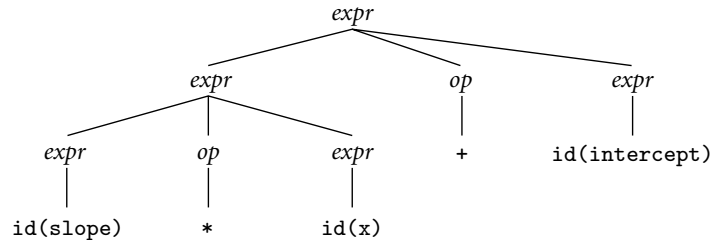


Figure 2.1 Parse tree for `slope * x + intercept` (grammar in Example 2.4).

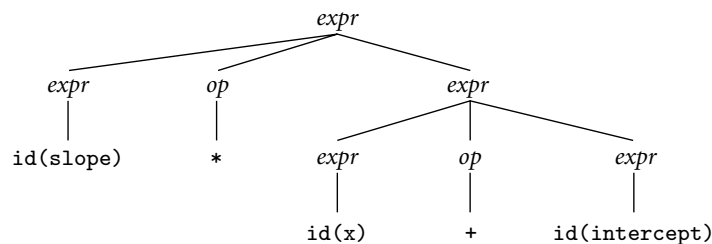


Figure 2.2 Alternative (less desirable) parse tree for `slope * x + intercept` (grammar in Example 2.4). The fact that more than one tree exists implies that our grammar is ambiguous.

EXAMPLE 2.7

Parse trees for `slope * x + intercept`

A parse tree for our example expression appears in Figure 2.1. This tree is not unique. At the second level of the tree, we could have chosen to turn the operator into a `*` instead of a `+`, and to further expand the expression on the right, rather than the one on the left (see Figure 2.2). A grammar that allows the construction of more than one parse tree for some string of terminals is said to be *ambiguous*. Ambiguity turns out to be a problem when trying to build a parser: it requires some extra mechanism to drive a choice between equally acceptable alternatives. ■

A moment's reflection will reveal that there are infinitely many context-free grammars for any given context-free language.⁹ Some grammars, however, are much more useful than others. In this text we will avoid the use of ambiguous grammars (though most parser generators allow them, by means of *disambiguating* rules). We will also avoid the use of so-called *useless* symbols: nonterminals that cannot generate any string of terminals, or terminals that cannot appear in the yield of any derivation.

When designing the grammar for a programming language, we generally try to find one that reflects the internal structure of programs in a way that is useful to the rest of the compiler. (We shall see in Section 2.3.2 that we also try to find one

⁹ Given a specific grammar, there are many ways to create other equivalent grammars. We could, for example, replace A with some new symbol B everywhere it appears in the right-hand side of a production, and then create a new production $B \rightarrow A$.

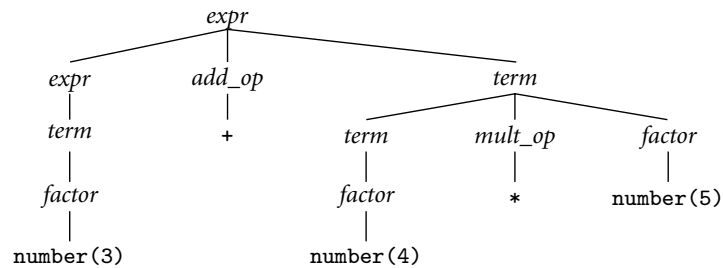


Figure 2.3 Parse tree for $3 + 4 * 5$, with precedence (grammar in Example 2.8).

that can be parsed efficiently, which can be a bit of a challenge.) One place in which structure is particularly important is in arithmetic expressions, where we can use productions to capture the *associativity* and *precedence* of the various operators. Associativity tells us that the operators in most languages group left to right, so that $10 - 4 - 3$ means $(10 - 4) - 3$ rather than $10 - (4 - 3)$. Precedence tells us that multiplication and division in most languages group more tightly than addition and subtraction, so that $3 + 4 * 5$ means $3 + (4 * 5)$ rather than $(3 + 4) * 5$. (These rules are not universal; we will consider them again in Section 6.1.1.)

EXAMPLE 2.8

Expression grammar with precedence and associativity

Here is a better version of our expression grammar:

1. $expr \rightarrow term \mid expr \text{ add_op } term$
2. $term \rightarrow factor \mid term \text{ mult_op } factor$
3. $factor \rightarrow id \mid number \mid - factor \mid (expr)$
4. $add_op \rightarrow + \mid -$
5. $mult_op \rightarrow * \mid /$

This grammar is unambiguous. It captures precedence in the way *factor*, *term*, and *expr* build on one another, with different operators appearing at each level. It captures associativity in the second halves of lines 1 and 2, which build sub*expr*s and sub*terms* to the left of the operator, rather than to the right. In Figure 2.3, we can see how building the notion of precedence into the grammar makes it clear that multiplication groups more tightly than addition in $3 + 4 * 5$, even without parentheses. In Figure 2.4, we can see that subtraction groups more tightly to the left, so that $10 - 4 - 3$ would evaluate to 3, rather than to 9. ■

✓ CHECK YOUR UNDERSTANDING

1. What is the difference between syntax and semantics?
2. What are the three basic operations that can be used to build complex regular expressions from simpler regular expressions?
3. What additional operation (beyond the three of regular expressions) is provided in context-free grammars?

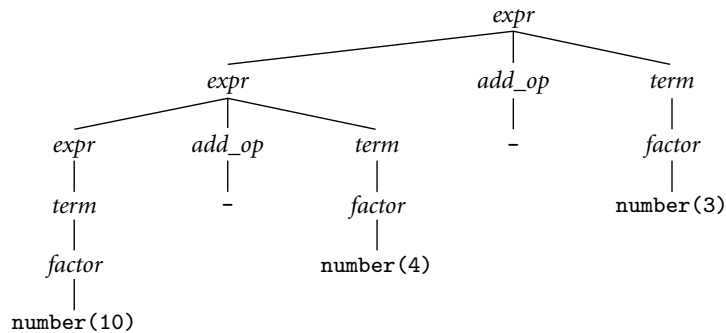


Figure 2.4 Parse tree for $10 - 4 - 3$, with left associativity (grammar in [Example 2.8](#)).

4. What is *Backus-Naur form*? When and why was it devised?
5. Name a language in which indentation affects program syntax.
6. When discussing context-free languages, what is a *derivation*? What is a *sentential form*?
7. What is the difference between a *right-most* derivation and a *left-most* derivation?
8. What does it mean for a context-free grammar to be *ambiguous*?
9. What are *associativity* and *precedence*? Why are they significant in parse trees?

2.2 Scanning

Together, the scanner and parser for a programming language are responsible for discovering the syntactic structure of a program. This process of discovery, or *syntax analysis*, is a necessary first step toward translating the program into an equivalent program in the target language. (It's also the first step toward interpreting the program directly. In general, we will focus on compilation, rather than interpretation, for the remainder of the book. Most of what we shall discuss either has an obvious application to interpretation, or is obviously irrelevant to it.)

By grouping input characters into tokens, the scanner dramatically reduces the number of individual items that must be inspected by the more computationally intensive parser. In addition, the scanner typically removes comments (so the parser doesn't have to worry about them appearing throughout the context-free grammar); saves the text of "interesting" tokens like identifiers, strings, and numeric literals; and tags tokens with line and column numbers, to make it easier to generate high-quality error messages in subsequent phases.

EXAMPLE 2.9

Tokens for a calculator language

In [Examples 2.4](#) and [2.8](#) we considered a simple language for arithmetic expressions. In [Section 2.3.1](#) we will extend this to create a simple “calculator language” with input, output, variables, and assignment. For this language we will use the following set of tokens.

```

assign  →  :=
plus   →  +
minus  →  -
times  →  *
div    →  /
lparen →  (
rparen →  )
id     →  letter ( letter | digit ) *
           except for read and write
number →  digit digit * | digit ( . digit | digit . ) digit *
```

In keeping with Algol and its descendants (and in contrast to the C-family languages), we have used `:=` rather than `=` for assignment. For simplicity, we have omitted the exponential notation found in [Example 2.3](#). We have also listed the tokens `read` and `write` as exceptions to the rule for `id` (more on this in [Section 2.2.2](#)). To make the task of the scanner a little more realistic, we borrow the two styles of comment from C:

```

comment →  /* ( non-* | * non-/ ) * */
           | // ( non-newline ) * newline
```

Here we have used *non-**, *non- /*, and *non-newline* as shorthand for the alternation of all characters other than `*`, `/`, and *newline*, respectively. ■

EXAMPLE 2.10

An ad hoc scanner for calculator tokens

How might we go about recognizing the tokens of our calculator language? The simplest approach is entirely ad hoc. Pseudocode appears in [Figure 2.5](#). We can structure the code however we like, but it seems reasonable to check the simpler

DESIGN & IMPLEMENTATION**Nested comments**

Nested comments can be handy for the programmer (e.g., for temporarily “commenting out” large blocks of code). Scanners normally deal only with non-recursive constructs, however, so nested comments require special treatment. Some languages disallow them. Others require the language implementor to augment the scanner with special-purpose comment-handling code. C++ and C99 strike a compromise: `/* . . . */` style comments are not allowed to nest, but `/* . . . */` and `// . . .` style comments can appear inside each other. The programmer can thus use one style for “normal” comments and the other for “commenting out.” (The C99 designers note, however, that conditional compilation (`#if`) is preferable [[Int03a](#), p. 58].)

```

skip any initial white space (spaces, tabs, and newlines)
if cur_char ∈ { '(', ')', '+', '-', '*' }
    return the corresponding single-character token
if cur_char = ':'
    read the next character
    if it is '=' then return assign else announce an error
if cur_char = '/'
    peek at the next character
    if it is '*' or '/'
        read additional characters until "*" or newline is seen, respectively
        jump back to top of code
    else return div
if cur_char = .
    read the next character
    if it is a digit
        read any additional digits
        return number
    else announce an error
if cur_char is a digit
    read any additional digits and at most one decimal point
    return number
if cur_char is a letter
    read any additional letters and digits
    check to see whether the resulting string is read or write
    if so then return the corresponding token
    else return id
else announce an error

```

Figure 2.5 Outline of an ad hoc scanner for tokens in our calculator language.

and more common cases first, to peek ahead when we need to, and to embed loops for comments and for long tokens such as identifiers and numbers.

After finding a token the scanner returns to the parser. When invoked again it repeats the algorithm from the beginning, using the next available characters of input (including any that were peeked at but not consumed the last time). ■

As a rule, we accept the longest possible token in each invocation of the scanner. Thus `foobar` is always `foobar` and never `f` or `foo` or `foob`. More to the point, in a language like C, `3.14159` is a real number and never `3`, `.`, and `14159`. White space (blanks, tabs, newlines, comments) is generally ignored, except to the extent that it separates tokens (e.g., `foo bar` is different from `foobar`).

Figure 2.5 could be extended fairly easily to outline a scanner for some larger programming language. The result could then be fleshed out, by hand, to create code in some implementation language. Production compilers often use such ad hoc scanners; the code is fast and compact. During development, however, it is usually preferable to build a scanner in a more structured way, as an explicit representation of a *finite automaton*. Finite automata can be generated automatically

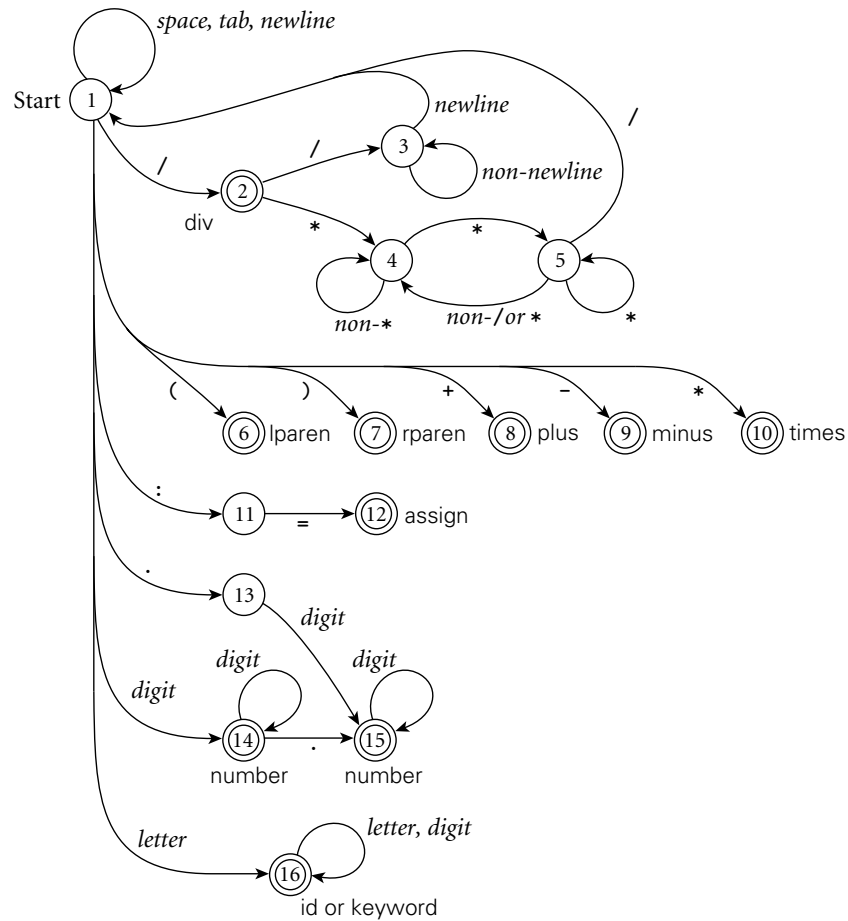


Figure 2.6 Pictorial representation of a scanner for calculator tokens, in the form of a finite automaton. This figure roughly parallels the code in Figure 2.5. States are numbered for reference in Figure 2.12. Scanning for each token begins in the state marked “Start.” The *final* states, in which a token is recognized, are indicated by double circles. Comments, when recognized, send the scanner back to its start state, rather than a final state.

from a set of regular expressions, making it easy to regenerate a scanner when token definitions change.

EXAMPLE 2.11

Finite automaton for a calculator scanner

An automaton for the tokens of our calculator language appears in pictorial form in Figure 2.6. The automaton starts in a distinguished initial state. It then moves from state to state based on the next available character of input. When it reaches one of a designated set of final states it recognizes the token associated with that state. The “longest possible token” rule means that the scanner returns to the parser only when the next character cannot be used to continue the current token. ■

2.2.1 Generating a Finite Automaton

While a finite automaton can in principle be written by hand, it is more common to build one automatically from a set of regular expressions, using a *scanner generator* tool. Because regular expressions are significantly easier to write and modify than is an ad hoc scanner, automatically generated scanners are often used during language or compiler development, or when ease of implementation is more important than the last little bit of run-time performance. In effect, regular expressions constitute a declarative programming language for a limited problem domain, namely that of scanning.

The example automaton of Figure 2.6 is *deterministic*: there is never any ambiguity about what it ought to do, because in a given state with a given input character there is never more than one possible outgoing transition (arrow) labeled by that character. As it turns out, however, there is no obvious one-step algorithm to convert a set of regular expressions into an equivalent deterministic finite automaton (DFA). The typical scanner generator implements the conversion as a series of three separate steps.

The first step converts the regular expressions into a *nondeterministic* finite automaton (NFA). An NFA is like a DFA except that (1) there may be more than one transition out of a given state labeled by a given character, and (2) there may be so-called *epsilon transitions*: arrows labeled by the empty string symbol, ϵ . The NFA is said to accept an input string (token) if there exists a path from the start state to a final state whose non-epsilon transitions are labeled, in order, by the characters of the token.

To avoid the need to search all possible paths for one that “works,” the second step of a scanner generator translates the NFA into an equivalent DFA: an automaton that accepts the same language, but in which there are no epsilon transitions, and no states with more than one outgoing transition labeled by the same character. The third step is a space optimization that generates a final DFA with the minimum possible number of states.

From a Regular Expression to an NFA

A trivial regular expression consisting of a single character c is equivalent to a simple two-state NFA (in fact, a DFA), illustrated in part (a) of Figure 2.7. Similarly, the regular expression ϵ is equivalent to a two-state NFA whose arc is labeled by ϵ . Starting with this base we can use three subconstructions, illustrated in parts (b) through (d) of the same figure, to build larger NFAs to represent the concatenation, alternation, or Kleene closure of the regular expressions represented by smaller NFAs. Each step preserves three invariants: there are no transitions into the initial state, there is a single final state, and there are no transitions out of the final state. These invariants allow smaller machines to be joined into larger machines without any ambiguity about where to create the connections, and without creating any unexpected paths. ■

EXAMPLE 2.12

Constructing an NFA for a given regular expression

EXAMPLE 2.13

NFA for $d^*(.d \mid d.)d^*$

To make these constructions concrete, we consider a small but nontrivial example—the *decimal* strings of Example 2.3. These consist of a string of decimal

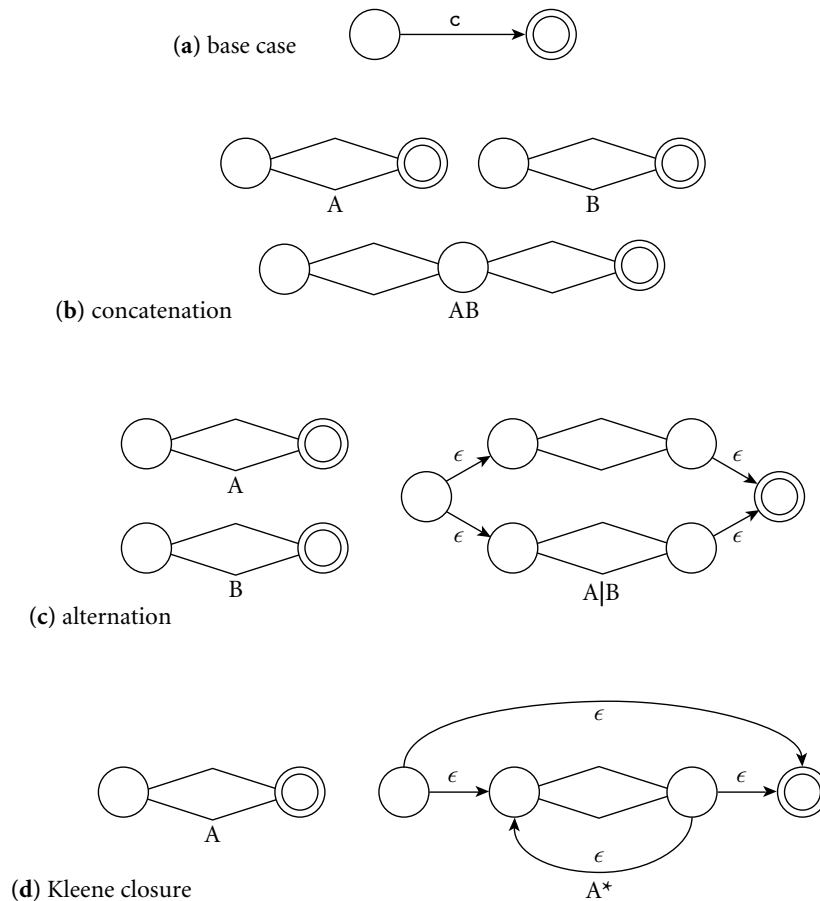


Figure 2.7 Construction of an NFA equivalent to a given regular expression. Part (a) shows the base case: the automaton for the single letter c . Parts (b), (c), and (d), respectively, show the constructions for concatenation, alternation, and Kleene closure. Each construction retains a unique start state and a single final state. Internal detail is hidden in the diamond-shaped center regions.

digits containing a single decimal point. With only one digit, the point can come at the beginning or the end: $(.d \mid d.)$, where for brevity we use d to represent any decimal digit. Arbitrary numbers of digits can then be added at the beginning or the end: $d^*(.d \mid d.)d^*$. Starting with this regular expression and using the constructions of Figure 2.7, we illustrate the construction of an equivalent NFA in Figure 2.8. ■

From an NFA to a DFA

EXAMPLE 2.14

DFA for $d^*(.d \mid d.)d^*$

With no way to “guess” the right transition to take from any given state, any practical implementation of an NFA would need to explore all possible transitions,

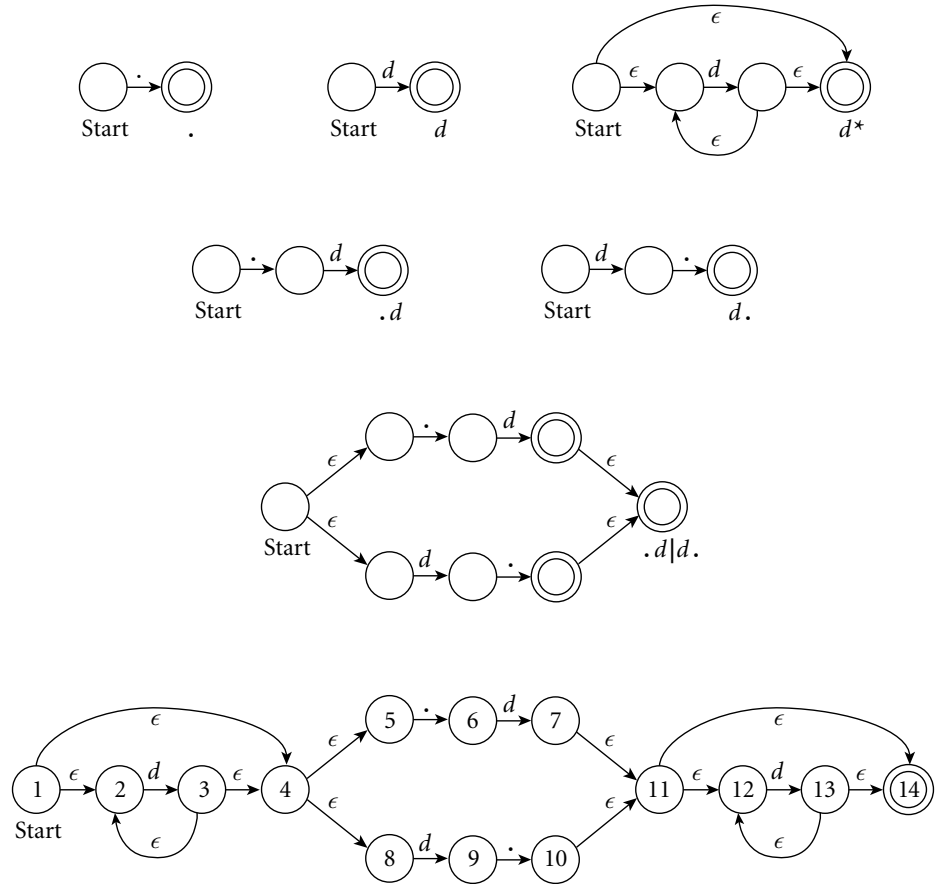


Figure 2.8 Construction of an NFA equivalent to the regular expression $d^*(.d|d.)d^*$. In the top row are the primitive automata for $.$ and d , and the Kleene closure construction for d^* . In the second and third rows we have used the concatenation and alternation constructions to build $.d$, $d.$, and $(.d|d.)$. The fourth row uses concatenation again to complete the NFA. We have labeled the states in the final automaton for reference in subsequent figures.

concurrently or via backtracking. To avoid such a complex and time-consuming strategy, we can use a “set of subsets” construction to transform the NFA into an equivalent DFA. The key idea is for the state of the DFA after reading a given input to represent the *set* of states that the NFA might have reached on the same input. We illustrate the construction in [Figure 2.9](#) using the NFA from [Figure 2.8](#). Initially, before it consumes any input, the NFA may be in State 1, or it may make epsilon transitions to States 2, 4, 5, or 8. We thus create an initial State *A* for our DFA to represent this set. On an input of d , our NFA may move from State 2 to State 3, or from State 8 to State 9. It has no other transitions on this input from any of the states in *A*. From State 3, however, the NFA may make epsilon transitions to any of States 2, 4, 5, or 8. We therefore create DFA State *B* as shown.

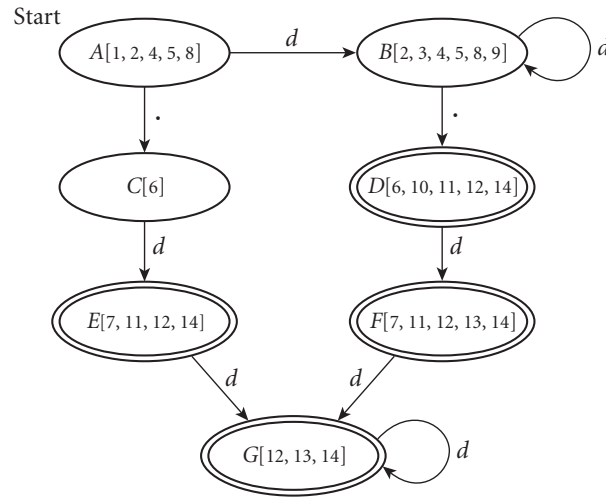


Figure 2.9 A DFA equivalent to the NFA at the bottom of Figure 2.8. Each state of the DFA represents the set of states that the NFA could be in after seeing the same input.

On a $.$, our NFA may move from State 5 to State 6. There are no other transitions on this input from any of the states in A , and there are no epsilon transitions out of State 6. We therefore create the singleton DFA State C as shown. None of States A , B , or C is marked as final, because none contains a final state of the original NFA.

Returning to State B of the growing DFA, we note that on an input of d the original NFA may move from State 2 to State 3, or from State 8 to State 9. From State 3, in turn, it may move to States 2, 4, 5, or 8 via epsilon transitions. As these are exactly the states already in B , we create a self-loop in the DFA. Given a $.$, on the other hand, the original NFA may move from State 5 to State 6, or from State 9 to State 10. From State 10, in turn, it may move to States 11, 12, or 14 via epsilon transitions. We therefore create DFA State D as shown, with a transition on $.$ from B to D . State D is marked as final because it contains state 14 of the original NFA. That is, given input $d.$, there exists a path from the start state to the end state of the original NFA. Continuing our enumeration of state sets, we end up creating three more, labeled E , F , and G in Figure 2.9. Like State D , these all contain State 14 of the original NFA, and thus are marked as final. ■

In our example, the DFA ends up being smaller than the NFA, but this is only because our regular language is so simple. In theory, the number of states in the DFA may be exponential in the number of states in the NFA, but this extreme is also uncommon in practice. For a programming language scanner, the DFA tends to be larger than the NFA, but not outlandishly so. We consider space complexity in more detail in Section 2.4.1.

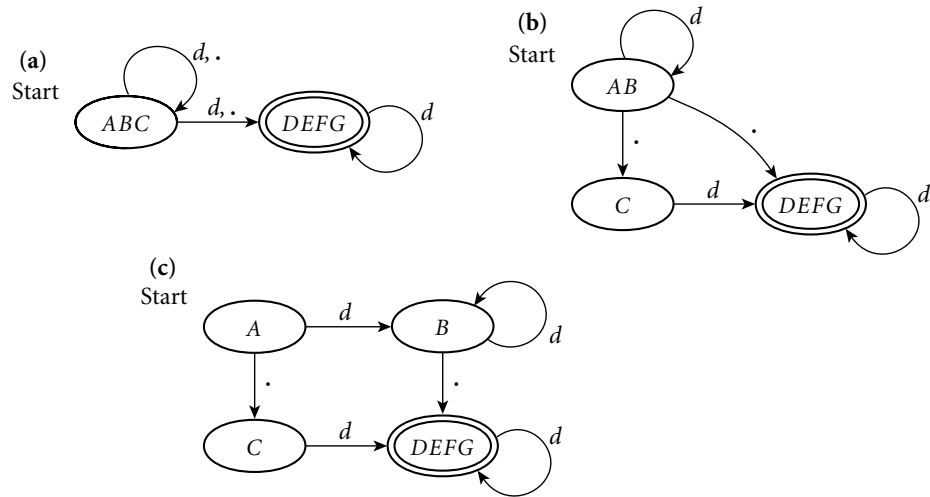


Figure 2.10 Minimization of the DFA of Figure 2.9. In each step we split a set of states to eliminate a transition ambiguity.

Minimizing the DFA

EXAMPLE 2.15

Minimal DFA for
 $d^*(.d \mid d.)d^*$

Starting from a regular expression we have now constructed an equivalent DFA. Though this DFA has seven states, a bit of thought suggests that a smaller one should exist. In particular, once we have seen both a *d* and a *.*, the only valid transitions are on *d*, and we ought to be able to make do with a single final state. We can formalize this intuition, allowing us to apply it to any DFA, via the following inductive construction.

Initially we place the states of the (not necessarily minimal) DFA into two equivalence classes: final states and nonfinal states. We then repeatedly search for an equivalence class $\mathcal{X}$ and an input symbol *c* such that when given *c* as input, the states in $\mathcal{X}$ make transitions to states in $k > 1$ different equivalence classes. We then partition $\mathcal{X}$ into *k* classes in such a way that all states in a given new class would move to a member of the same old class on *c*. When we are unable to find a class to partition in this fashion we are done.

In our example, the original placement puts States *D*, *E*, *F*, and *G* in one class (final states) and States *A*, *B*, and *C* in another, as shown in the upper left of Figure 2.10. Unfortunately, the start state has ambiguous transitions on both *d* and *.*. To address the *d* ambiguity, we split *ABC* into *AB* and *C*, as shown in the upper right. New State *AB* has a self-loop on *d*; new State *C* moves to State *DEFG*. State *AB* still has an ambiguity on *.*, however, which we resolve by splitting it into States *A* and *B*, as shown at the bottom of the figure. At this point there are no further ambiguities, and we are left with a four-state minimal DFA. ■

2.2.2 Scanner Code

We can implement a scanner that explicitly captures the “circles-and-arrows” structure of a DFA in either of two main ways. One embeds the automaton in the control flow of the program using `gotos` or nested `case` (`switch`) statements; the other, described in the following subsection, uses a table and a driver. As a general rule, handwritten automata tend to use nested case statements, while most (but not all [BC93]) automatically generated automata use tables. Tables are hard to create by hand, but easier than code to create from within a program. Likewise, nested case statements are easier to write and to debug than the ad hoc approach of Figure 2.5, if not quite as efficient. Unix’s `lex/flex` tool produces C language output containing tables and a customized driver.

EXAMPLE 2.16

Nested case statement automaton

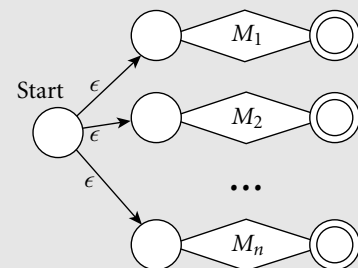
The nested case statement style of automaton has the following general structure.

DESIGN & IMPLEMENTATION

Recognizing multiple kinds of token

One of the chief ways in which a scanner differs from a formal DFA is that it *identifies* tokens in addition to recognizing them. That is, it not only determines whether characters constitute a valid token; it also indicates *which one*. In practice, this means that it must have separate final states for every kind of token. We glossed over this issue in our RE-to-DFA constructions.

To build a scanner for a language with n different kinds of tokens, we begin with an NFA of the sort suggested in the figure here. Given NFAs M_i , $1 \leq i \leq n$ (one machine for each kind of token), we create a new start state with epsilon transitions to the start states of the M_i s. In contrast to the alternation construction of Figure 2.7(c), however, we do *not* create a single final state; we keep the existing ones, each labeled by the token for which it is final. We then apply the NFA-to-DFA construction as before. (If final states for different tokens in the NFA ever end up in the same state of the DFA, then we have ambiguous token definitions. These may be resolved by changing the regular expressions from which the NFAs were derived, or by wrapping additional logic around the DFA.)



In the DFA minimization construction, instead of starting with two equivalence classes (final and nonfinal states), we begin with $n+1$, including a separate class for final states for each of the kinds of token. Exercise 2.5 explores this construction for a scanner that recognizes both the *integer* and *decimal* types of Example 2.3.

```

state := 1          -- start state
loop
  read cur_char
  case state of
    1 : case cur_char of
      ' ', '\t', '\n' : ...
      'a'...'z' : ...
      '0'...'9' : ...
      '>' : ...
      ...
    2 : case cur_char of
      ...
      ...
    n: case cur_char of
      ...

```

The outer case statement covers the states of the finite automaton. The inner case statements cover the transitions out of each state. Most of the inner clauses simply set a new state. Some return from the scanner with the current token. (If the current character should not be part of that token, it is pushed back onto the input stream before returning.) ■

Two aspects of the code typically deviate from the strict form of a formal finite automaton. One is the handling of keywords. The other is the need to peek ahead when a token can validly be extended by two or more additional characters, but not by only one.

As noted at the beginning of [Section 2.1.1](#), keywords in most languages look just like identifiers, but are reserved for a special purpose (some authors use the term *reserved word* instead of keyword¹⁰). It is possible to write a finite automaton that distinguishes between keywords and identifiers, but it requires a *lot* of states (see [Exercise 2.3](#)). Most scanners, both handwritten and automatically generated, therefore treat keywords as “exceptions” to the rule for identifiers. Before returning an identifier to the parser, the scanner looks it up in a hash table or trie (a tree of branching paths) to make sure it isn’t really a keyword.

Whenever one legitimate token is a prefix of another, the “longest possible token” rule says that we should continue scanning. If some of the intermediate strings are not valid tokens, however, we can’t tell whether a longer token is possible without looking more than one character ahead. This problem arises with dot characters (periods) in C. Suppose the scanner has just seen a 3 and has a dot coming up in the input. It needs to peek at characters beyond the dot in order to distinguish between 3.14 (a single token designating a real number), 3 . foo

EXAMPLE 2.17

The nontrivial prefix problem

10 Keywords (reserved words) are not the same as *predefined* identifiers. Predefined identifiers can be redefined to have a different meaning; keywords cannot. The scanner does not distinguish between predefined and other identifiers. It does distinguish between identifiers and keywords. C doesn’t really have any predefined identifiers, but many languages do. In Pascal, for example, the names of built-in types and standard library functions are predefined but not reserved.

(three tokens that the scanner should accept, even though the parser will object to seeing them in that order), and `3 . . . foo` (again not syntactically valid, but three separate tokens nonetheless). In general, upcoming characters that a scanner must examine in order to make a decision are known as its *look-ahead*. In [Section 2.3](#) we will see a similar notion of look-ahead *tokens* in parsing. ■

EXAMPLE 2.18

Look-ahead in Fortran
scanning

In messier languages, a scanner may need to look an arbitrary distance ahead. In Fortran IV, for example, `D0 5 I = 1, 25` is the header of a loop (it executes the statements up to the one labeled 5 for values of `I` from 1 to 25), while `D0 5 I = 1.25` is an assignment statement that places the value 1.25 into the variable `D05I`. Spaces are ignored in (pre-Fortran 90) Fortran input, even in the middle of variable names. Moreover, variables need not be declared, and the terminator for a `D0` loop is simply a label, which the parser can ignore. After seeing `D0`, the scanner cannot tell whether the 5 is part of the current token until it reaches the comma or dot. It has been widely (but apparently incorrectly) claimed that NASA's Mariner 1 space probe was lost due to accidental replacement of a comma with a dot in a case similar to this one in flight control software.¹¹ Dialects of Fortran starting with Fortran 77 allow (in fact encourage) the use of alternative syntax for loop headers, in which an extra comma makes misinterpretation less likely: `D0 5, I = 1, 25`. ■

In C, the the dot character problem can easily be handled as a special case. In languages requiring larger amounts of look-ahead, the scanner can take a more general approach. In any case of ambiguity, it assumes that a longer token will be possible, but remembers that a shorter token could have been recognized at some point in the past. It also buffers all characters read beyond the end of the shorter token. If the optimistic assumption leads the scanner into an error state, it “unreads” the buffered characters so that they will be seen again later, and returns the shorter token.

DESIGN & IMPLEMENTATION**Longest possible tokens**

A little care in syntax design—avoiding tokens that are nontrivial prefixes of other tokens—can dramatically simplify scanning. In straightforward cases of prefix ambiguity the scanner can enforce the “longest possible token” rule automatically. In Fortran, however, the rules are sufficiently complex that no purely lexical solution suffices. Some of the problems, and a possible solution, are discussed in an article by Dyadkin [[Dya95](#)].

11 In actuality, the faulty software for Mariner 1 appears to have stemmed from a missing “bar” punctuation mark (indicating an average) in handwritten notes from which the software was derived [[Cer89](#), pp. 202–203]. The Fortran `D0` loop error does appear to have occurred in at least one piece of NASA software, but no serious harm resulted [[Web89](#)].

2.2.3 Table-Driven Scanning

EXAMPLE 2.19

Table-driven scanning

In the preceding subsection we sketched how control flow—a loop and nested case statements—can be used to represent a finite automaton. An alternative approach represents the automaton as a data structure: a two-dimensional *transition table*. A driver program (Figure 2.11) uses the current state and input character to index into the table. Each entry in the table specifies whether to move to a new state (and if so, which one), return a token, or announce an error. A second table indicates, for each state, whether we might be at the end of a token (and if so, which one). Separating this second table from the first allows us to notice when we pass a state that might have been the end of a token, so we can back up if we hit an error state. Example tables for our calculator tokens appear in Figure 2.12.

Like a handwritten scanner, the table-driven code of Figure 2.11 looks tokens up in a table of keywords immediately before returning. An outer loop serves to filter out comments and “white space”—spaces, tabs, and newlines. These character sequences are not meaningful to the parser, and would in fact be very difficult to represent in a grammar (Exercise 2.20). ■

2.2.4 Lexical Errors

The code in Figure 2.11 explicitly recognizes the possibility of *lexical errors*. In some cases the next character of input may be neither an acceptable continuation of the current token nor the start of another token. In such cases the scanner must print an error message and perform some sort of recovery so that compilation can continue, if only to look for additional errors. Fortunately, lexical errors are relatively rare—most character sequences do correspond to token sequences—and relatively easy to handle. The most common approach is simply to (1) throw away the current, invalid token; (2) skip forward until a character is found that can legitimately begin a new token; (3) restart the scanning algorithm; and (4) count on the error-recovery mechanism of the parser to cope with any cases in which the resulting sequence of tokens is not syntactically valid. Of course the need for error recovery is not unique to table-driven scanners; any scanner must cope with errors. We did not show the code in Figure 2.5, but it would have to be there in practice.

The code in Figure 2.11 also shows that the scanner must return both the kind of token found and its character-string image (spelling); again this requirement applies to all types of scanners. For some tokens the character-string image is redundant: all semicolons look the same, after all, as do all `while` keywords. For other tokens, however (e.g., identifiers, character strings, and numeric constants), the image is needed for semantic analysis. It is also useful for error messages: “undeclared identifier” is not as nice as “foo has not been declared.”

```

state = 0 .. number_of_states
token = 0 .. number_of_tokens
scan_tab : array [char, state] of record
    action : (move, recognize, error)
    new_state : state
token_tab : array [state] of token      -- what to recognize
keyword_tab : set of record
    k_image : string
    k_token : token
-- these three tables are created by a scanner generator tool

tok : token
cur_char : char
remembered_chars : list of char
repeat
    cur_state : state := start_state
    image : string := null
    remembered_state : state := 0      -- none
    loop
        read cur_char
        case scan_tab[cur_char, cur_state].action
        move:
            if token_tab[cur_state] ≠ 0
                -- this could be a final state
                remembered_state := cur_state
                remembered_chars := ε
                add cur_char to remembered_chars
                cur_state := scan_tab[cur_char, cur_state].new_state
            recognize:
                tok := token_tab[cur_state]
                unread cur_char      -- push back into input stream
                exit inner loop
            error:
                if remembered_state ≠ 0
                    tok := token_tab[remembered_state]
                    unread remembered_chars
                    remove remembered_chars from image
                    exit inner loop
                -- else print error message and recover; probably start over
            append cur_char to image
        -- end inner loop
    until tok ∉ {white_space, comment}
    look image up in keyword_tab and replace tok with appropriate keyword if found
    return ⟨tok, image⟩

```

Figure 2.11 Driver for a table-driven scanner, with code to handle the ambiguous case in which one valid token is a prefix of another, but some intermediate string is not.

State	Current input character														
	<i>space, tab</i>	<i>newline</i>	<i>/</i>	<i>*</i>	<i>(</i>	<i>)</i>	<i>+</i>	<i>-</i>	<i>:</i>	<i>=</i>	<i>.</i>	<i>digit</i>	<i>letter</i>	<i>other</i>	
1	17	17	2	10	6	7	8	9	11	—	13	14	16	—	div
2	—	—	3	4	—	—	—	—	—	—	—	—	—	—	
3	3	18	3	3	3	3	3	3	3	3	3	3	3	3	
4	4	4	4	5	4	4	4	4	4	4	4	4	4	4	
5	4	4	18	5	4	4	4	4	4	4	4	4	4	4	lparen
6	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
7	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
8	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
9	—	—	—	—	—	—	—	—	—	—	—	—	—	—	rparen
10	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
11	—	—	—	—	—	—	—	—	—	12	—	—	—	—	
12	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
13	—	—	—	—	—	—	—	—	—	—	—	15	—	—	plus
14	—	—	—	—	—	—	—	—	—	—	15	14	—	—	
15	—	—	—	—	—	—	—	—	—	—	—	15	—	—	
16	—	—	—	—	—	—	—	—	—	—	—	16	16	—	
17	17	17	—	—	—	—	—	—	—	—	—	—	—	—	minus
18	—	—	—	—	—	—	—	—	—	—	—	—	—	—	

Figure 2.12 Scanner tables for the calculator language. These could be used by the code of Figure 2.11. States are numbered as in Figure 2.6, except for the addition of two states—17 and 18—to “recognize” white space and comments. The right-hand column represents table `token.tab`; the rest of the figure is `scan.tab`. Dashes indicate no way to extend the current token. Table `keyword.tab` (not shown) contains the strings `read` and `write`.

2.2.5 Pragmas

Some languages and language implementations allow a program to contain constructs called *pragmas* that provide directives or hints to the compiler. Pragmas that do not change program semantics—only the compilation process—are sometimes called *significant comments*. In some languages the name is also appropriate because, like comments, pragmas can appear anywhere in the source program. In this case they are usually processed by the scanner: allowing them anywhere in the grammar would greatly complicate the parser. In other languages (Ada, for example), pragmas are permitted only at certain well-defined places in the grammar. In this case they are best processed by the parser or semantic analyzer.

Pragmas that serve as directives may:

- Turn various kinds of run-time checks (e.g., pointer or subscript checking) on or off
- Turn certain code improvements on or off (e.g., on in inner loops to improve performance; off otherwise to improve compilation speed)

- Enable or disable performance profiling (statistics gathering to identify program bottlenecks)

Some directives “cross the line” and change program semantics. In Ada, for example, the `unchecked pragma` can be used to disable type checking. In OpenMP, which we will consider in [Chapter 12](#), pragmas specify significant parallel extensions to Fortran, C and C++: creating, scheduling, and synchronizing threads. In this case the principal rationale for expressing the extensions as pragmas rather than more deeply integrated changes is to sharply delineate the boundary between the core language and the extensions, and to share a common set of extensions across languages.

Pragmas that serve (merely) as hints provide the compiler with information about the source program that may allow it to do a better job:

- Variable `x` is very heavily used (it may be a good idea to keep it in a register).
- Subroutine `F` is a pure function: its only effect on the rest of the program is the value it returns.
- Subroutine `S` is not (indirectly) recursive (its storage may be statically allocated).
- 32 bits of precision (instead of 64) suffice for floating-point variable `x`.

The compiler may ignore these in the interest of simplicity, or in the face of contradictory information.

✓ CHECK YOUR UNDERSTANDING

10. List the tasks performed by the typical scanner.
 11. What are the advantages of an automatically generated scanner, in comparison to a handwritten one? Why do many commercial compilers use a handwritten scanner anyway?
 12. Explain the difference between deterministic and nondeterministic finite automata. Why do we prefer the deterministic variety for scanning?
 13. Outline the constructions used to turn a set of regular expressions into a minimal DFA.
 14. What is the “longest possible token” rule?
 15. Why must a scanner sometimes “peek” at upcoming characters?
 16. What is the difference between a *keyword* and an *identifier*?
 17. Why must a scanner save the text of tokens?
 18. How does a scanner identify lexical errors? How does it respond?
 19. What is a *pragma*?
-

2.3 Parsing

The parser is the heart of a typical compiler. It calls the scanner to obtain the tokens of the input program, assembles the tokens together into a syntax tree, and passes the tree (perhaps one subroutine at a time) to the later phases of the compiler, which perform semantic analysis and code generation and improvement. In effect, the parser is “in charge” of the entire compilation process; this style of compilation is sometimes referred to as *syntax-directed translation*.

As noted in the introduction to this chapter, a context-free grammar (CFG) is a *generator* for a CF language. A parser is a language *recognizer*. It can be shown that for any CFG we can create a parser that runs in $O(n^3)$ time, where n is the length of the input program.¹² There are two well-known parsing algorithms that achieve this bound: Earley’s algorithm [Ear70] and the Cocke-Younger-Kasami (CYK) algorithm [Kas65, You67]. Cubic time is much too slow for parsing sizable programs, but fortunately not all grammars require such a general and slow parsing algorithm. There are large classes of grammars for which we can build parsers that run in linear time. The two most important of these classes are called LL and LR.

LL stands for “Left-to-right, Left-most derivation.” LR stands for “Left-to-right, Right-most derivation.” In both classes the input is read left-to-right, and the parser attempts to discover (construct) a derivation of that input. For LL parsers, the derivation will be left-most; for LR parsers, right-most. We will cover LL parsers first. They are generally considered to be simpler and easier to understand. They can be written by hand or generated automatically from an appropriate grammar by a parser-generating tool. The class of LR grammars is larger (i.e., more grammars are LR than LL), and some people find the structure of the LR grammars more intuitive, especially in the handling of arithmetic expressions. LR parsers are almost always constructed by a parser-generating tool. Both classes of parsers are used in production compilers, though LR parsers are more common.

LL parsers are also called “top-down,” or “predictive” parsers. They construct a parse tree from the root down, predicting at each step which production will be used to expand the current node, based on the next available token of input. LR parsers are also called “bottom-up” parsers. They construct a parse tree from the leaves up, recognizing when a collection of leaves or other nodes can be joined together as the children of a single parent.

We can illustrate the difference between top-down and bottom-up parsing by means of a simple example. Consider the following grammar for a comma-separated list of identifiers, terminated by a semicolon:

$$id_list \longrightarrow id\ id_list_tail$$

EXAMPLE 2.20

Top-down and bottom-up parsing

¹² In general, an algorithm is said to run in time $O(f(n))$, where n is the length of the input, if its running time $t(n)$ is proportional to $f(n)$ in the worst case. More precisely, we say $t(n) = O(f(n)) \iff \exists c, m [n > m \implies t(n) < cf(n)]$.

$$\begin{aligned} id_list_tail &\longrightarrow , id id_list_tail \\ id_list_tail &\longrightarrow ; \end{aligned}$$

These are the productions that would normally be used for an identifier list in a top-down parser. They can also be parsed bottom-up (most top-down grammars can be). In practice they would not be used in a bottom-up parser, for reasons that will become clear in a moment, but the ability to handle them either way makes them good for this example.

Progressive stages in the top-down and bottom-up construction of a parse tree for the string *A, B, C;* appear in [Figure 2.13](#). The top-down parser begins by predicting that the root of the tree (*id_list*) will be replaced by *id id_list_tail*. It then matches the *id* against a token obtained from the scanner. (If the scanner produced something different, the parser would announce a syntax error.) The parser then moves down into the first (in this case only) nonterminal child and predicts that *id_list_tail* will be replaced by *, id id_list_tail*. To make this prediction it needs to peek at the upcoming token (a comma), which allows it to choose between the two possible expansions for *id_list_tail*. It then matches the comma and the *id* and moves down into the next *id_list_tail*. In a similar, recursive fashion, the top-down parser works down the tree, left-to-right, predicting and expanding nodes and tracing out a left-most derivation of the fringe of the tree.

The bottom-up parser, by contrast, begins by noting that the left-most leaf of the tree is an *id*. The next leaf is a comma and the one after that is another *id*. The parser continues in this fashion, shifting new leaves from the scanner into a forest of partially completed parse tree fragments, until it realizes that some of those fragments constitute a complete right-hand side. In this grammar, that doesn't occur until the parser has seen the semicolon—the right-hand side of *id_list_tail* $\longrightarrow ;$. With this right-hand side in hand, the parser reduces the semicolon to an *id_list_tail*. It then reduces *, id id_list_tail* into another *id_list_tail*. After doing this one more time it is able to reduce *id id_list_tail* into the root of the parse tree, *id_list*.

At no point does the bottom-up parser predict what it will see next. Rather, it shifts tokens into its forest until it recognizes a right-hand side, which it then reduces to a left-hand side. Because of this behavior, bottom-up parsers are sometimes called shift-reduce parsers. Moving up the figure, from bottom to top, we can see that the shift-reduce parser traces out a right-most derivation, in reverse. Because bottom-up parsers were the first to receive careful formal study, right-most derivations are sometimes called *canonical*. ■

There are several important subclasses of LR parsers, including SLR, LALR, and “full LR.” SLR and LALR are important for their ease of implementation, full LR for its generality. LL parsers can also be grouped into SLL and “full LL” subclasses. We will cover the differences among them only briefly here; for further information see any of the standard compiler-construction or parsing theory textbooks [[App97](#), [ALSU07](#), [AU72](#), [CT04](#), [FL88](#)].

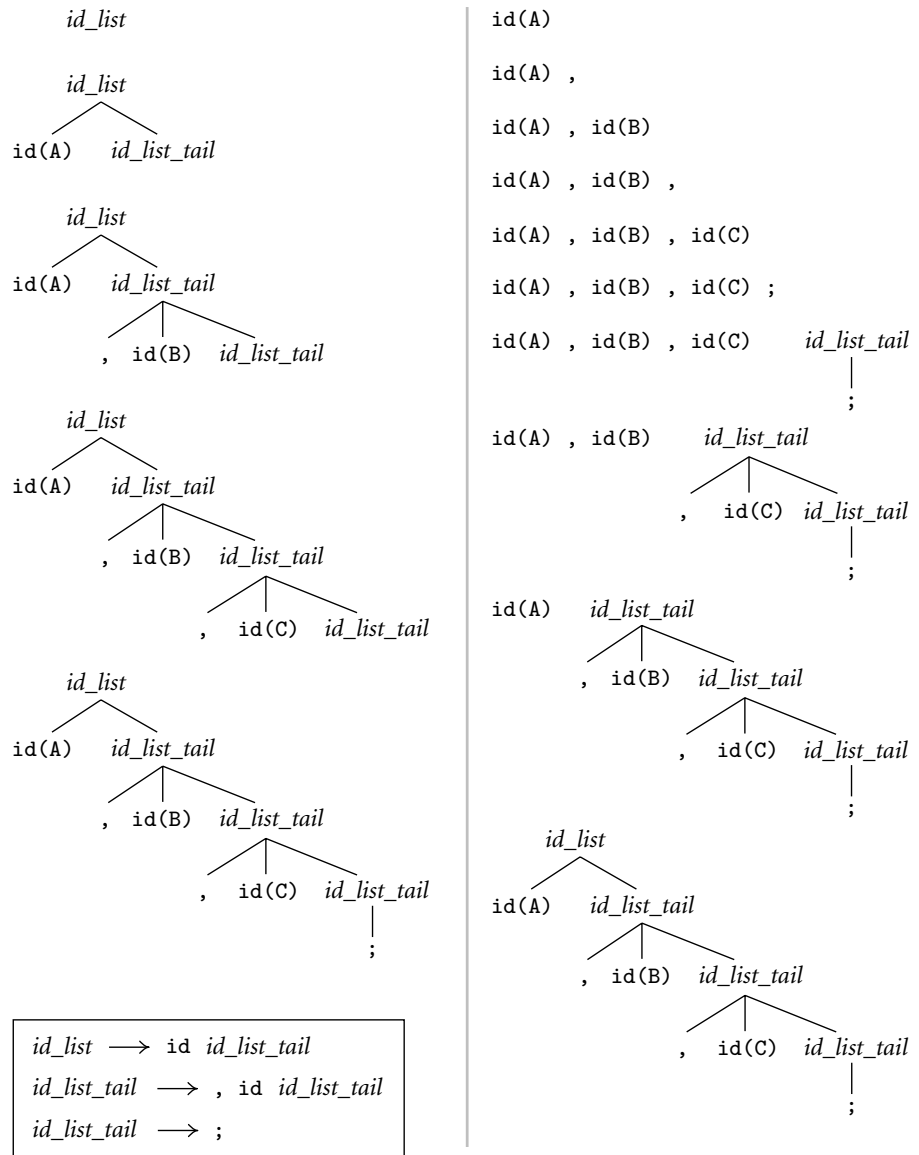


Figure 2.13 Top-down (left) and bottom-up parsing (right) of the input string `A, B, C;`. Grammar appears at lower left.

One commonly sees LL or LR (or whatever) written with a number in parentheses after it: LL(2) or LALR(1), for example. This number indicates how many tokens of look-ahead are required in order to parse. Most real compilers use just one token of look-ahead, though more can sometimes be helpful. Terrence Parr's open-source ANTLR tool, in particular, uses multitoken look-ahead to enlarge

the class of languages amenable to top-down parsing [PQ95]. In Section 2.3.1 we will look at LL(1) grammars and handwritten parsers in more detail. In Sections 2.3.2 and 2.3.3 we will consider automatically generated LL(1) and LR(1) (actually SLR(1)) parsers.

EXAMPLE 2.21

Bounding space with a bottom-up grammar

The problem with our example grammar, for the purposes of bottom-up parsing, is that it forces the compiler to shift all the tokens of an *id_list* into its forest before it can reduce any of them. In a very large program we might run out of space. Sometimes there is nothing that can be done to avoid a lot of shifting. In this case, however, we can use an alternative grammar that allows the parser to reduce prefixes of the *id_list* into nonterminals as it goes along:

$$\begin{aligned} id_list &\longrightarrow id_list_prefix \ ; \\ id_list_prefix &\longrightarrow id_list_prefix \ , \ id \\ &\longrightarrow id \end{aligned}$$

This grammar cannot be parsed top-down, because when we see an *id* on the input and we're expecting an *id_list_prefix*, we have no way to tell which of the two possible productions we should predict (more on this dilemma in Section 2.3.2). As shown in Figure 2.14, however, the grammar works well bottom-up. ■

2.3.1 Recursive Descent

EXAMPLE 2.22

Top-down grammar for a calculator language

To illustrate top-down (predictive) parsing, let us consider the grammar for a simple “calculator” language, shown in Figure 2.15. The calculator allows values to be read into (numeric) variables, which may then be used in expressions. Expressions in turn can be written to the output. Control flow is strictly linear (no loops, *if* statements, or other jumps). The end-marker (*\$\$*) pseudotoken is produced by the scanner at the end of the input. This token allows the parser to terminate cleanly once it has seen the entire program. As in regular expressions, we use the symbol ϵ to denote the empty string. A production with ϵ on the right-hand side is sometimes called an *epsilon production*.

It may be helpful to compare the *expr* portion of Figure 2.15 to the expression grammar of Example 2.8 (page 50). Most people find that previous, LR grammar to be significantly more intuitive. It suffers, however, from a problem similar to that of the *id_list* grammar of Example 2.21: if we see an *id* on the input when expecting an *expr*, we have no way to tell which of the two possible productions to predict. The grammar of Figure 2.15 avoids this problem by merging the common prefixes of right-hand sides into a single production, and by using new symbols (*term_tail* and *factor_tail*) to generate additional operators and operands as required. The transformation has the unfortunate side effect of placing the operands of a given operator in separate right-hand sides. In effect, we have sacrificed grammatical elegance in order to be able to parse predictively. ■

So how do we parse a string with our calculator grammar? We saw the basic idea in Figure 2.13. We start at the top of the tree and predict needed productions on the basis of the current left-most nonterminal in the tree and the current

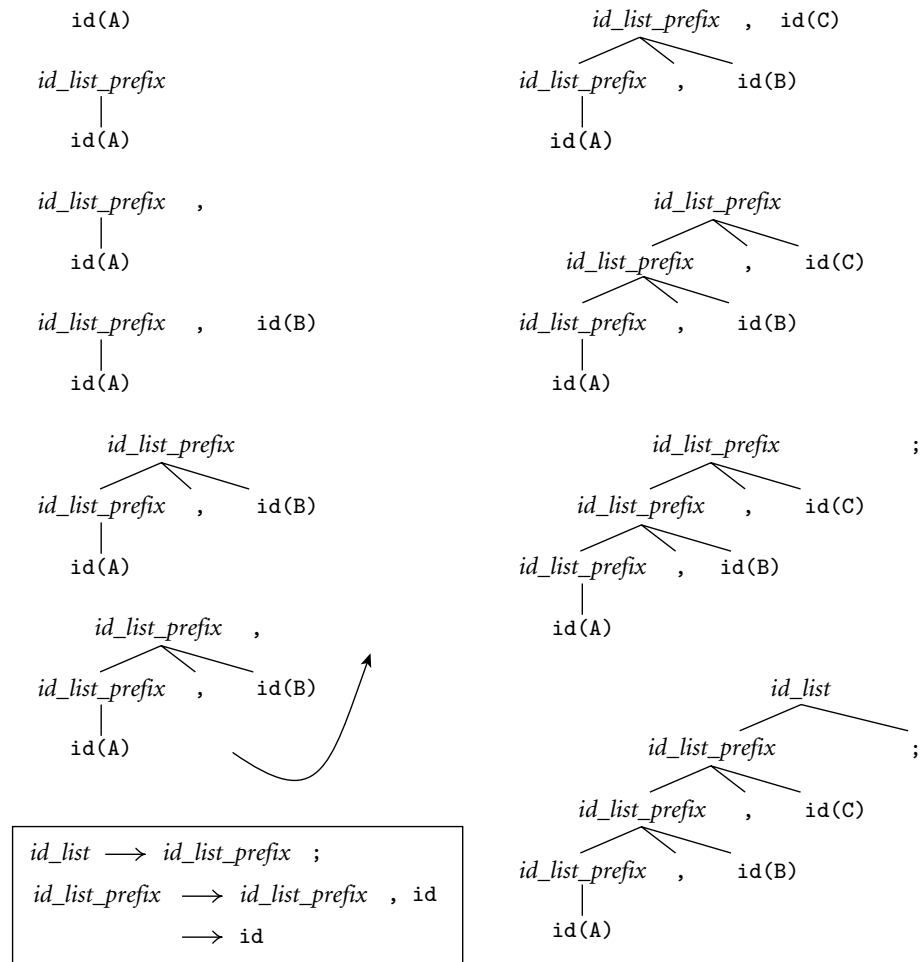


Figure 2.14 Bottom-up parse of A, B, C; using a grammar (lower left) that allows lists to be collapsed incrementally.

input token. We can formalize this process in one of two ways. The first, described in the remainder of this subsection, is to build a *recursive descent parser* whose subroutines correspond, one-one, to the nonterminals of the grammar. Recursive descent parsers are typically constructed by hand, though the ANTLR parser generator constructs them automatically from an input grammar. The second approach, described in [Section 2.3.2](#), is to build an *LL parse table* which is then read by a driver program. Table-driven parsers are almost always constructed automatically by a parser generator. These two options—recursive descent and table-driven—are reminiscent of the nested case statements and table-driven approaches to building a scanner that we saw in [Sections 2.2.2](#) and [2.2.3](#). It should be emphasized that they implement the same basic parsing algorithm.

```

program  $\rightarrow$  stmt_list $$
stmt_list  $\rightarrow$  stmt stmt_list |  $\epsilon$ 
stmt  $\rightarrow$  id := expr | read id | write expr
expr  $\rightarrow$  term term_tail
term_tail  $\rightarrow$  add_op term term_tail |  $\epsilon$ 
term  $\rightarrow$  factor factor_tail
factor_tail  $\rightarrow$  mult_op factor factor_tail |  $\epsilon$ 
factor  $\rightarrow$  ( expr ) | id | number
add_op  $\rightarrow$  + | -
mult_op  $\rightarrow$  * | /

```

Figure 2.15 LL(1) grammar for a simple calculator language.

Handwritten recursive descent parsers are most often used when the language to be parsed is relatively simple, or when a parser-generator tool is not available. There are exceptions, however. In particular, recursive descent appears in recent versions of the GNU compiler collection (`gcc`). Earlier versions used `bison` to create a bottom-up parser automatically. The change was made in part for performance reasons and in part to enable the generation of higher-quality syntax error messages. (The `bison` code was easier to write, and arguably easier to maintain.)

EXAMPLE 2.23

Recursive descent parser
for the calculator language

Pseudocode for a recursive descent parser for our calculator language appears in Figure 2.16. It has a subroutine for every nonterminal in the grammar. It also has a mechanism `input_token` to inspect the next token available from the scanner and a subroutine (`match`) to consume and update this token, and in the process verify that it is the one that was expected (as specified by an argument). If `match` or any of the other subroutines sees an unexpected token, then a syntax error has occurred. For the time being let us assume that the `parse_error` subroutine simply prints a message and terminates the parse. In Section 2.3.4 we will consider how to recover from such errors and continue to parse the remainder of the input. ■

EXAMPLE 2.24

Recursive descent parse of
a “sum and average”
program

Suppose now that we are to parse a simple program to read two numbers and print their sum and average:

```

read A
read B
sum := A + B
write sum
write sum / 2

```

The parse tree for this program appears in Figure 2.17. The parser begins by calling the subroutine `program`. After noting that the initial token is a `read`,

program calls `stmt_list` and then attempts to match the end-of-file pseudotoken. (In the parse tree, the root, *program*, has two children, *stmt_list* and `$$`.) Procedure `stmt_list` again notes that the upcoming token is a read. This observation allows it to determine that the current node (*stmt_list*) generates *stmt stmt_list* (rather than ϵ). It therefore calls `stmt` and `stmt_list` before returning. Continuing in this fashion, the execution path of the parser traces out a left-to-right depth-first traversal of the parse tree. This correspondence between the dynamic execution trace and the structure of the parse tree is the distinguishing characteristic of recursive descent parsing. Note that because the *stmt_list* non-terminal appears in the right-hand side of a *stmt_list* production, the `stmt_list` subroutine must call itself. This recursion accounts for the name of the parsing technique. ■

Without additional code (not shown in Figure 2.16), the parser merely verifies that the program is syntactically correct (i.e., that none of the otherwise `parse_error` clauses in the case statements are executed and that `match` always sees what it expects to see). To be of use to the rest of the compiler—which must produce an equivalent target program in some other language—the parser must save the parse tree or some other representation of program fragments as an explicit data structure. To save the parse tree itself, we can allocate and link together records to represent the children of a node immediately before executing the recursive subroutines and `match` invocations that represent those children. We shall need to pass each recursive routine an argument that points to the record that is to be expanded (i.e., whose children are to be discovered). Procedure `match` will also need to save information about certain tokens (e.g., character-string representations of identifiers and literals) in the leaves of the tree.

As we saw in Chapter 1, the parse tree contains a great deal of irrelevant detail that need not be saved for the rest of the compiler. It is therefore rare for a parser to construct a full parse tree explicitly. More often it produces an abstract syntax tree or some other more terse representation. In a recursive descent compiler, a syntax tree can be created by allocating and linking together records in only a subset of the recursive calls.

The trickiest part of writing a recursive descent parser is figuring out which tokens should label the arms of the case statements. Each arm represents one production: one possible expansion of the symbol for which the subroutine was named. The tokens that label a given arm are those that *predict* the production. A token *X* may predict a production for either of two reasons: (1) the right-hand side of the production, when recursively expanded, may yield a string beginning with *X*, or (2) the right-hand side may yield nothing (i.e., it is ϵ , or a string of nonterminals that may recursively yield ϵ), and *X* may begin the yield of what comes *next*. In the following subsection we will formalize this notion of prediction using sets called `FIRST` and `FOLLOW`, and show how to derive them automatically from an LL(1) CFG.

```

procedure match(expected)
    if input_token = expected then consume input_token
    else parse_error

-- this is the start routine:
procedure program
    case input_token of
        id, read, write, $$ :
            stmt_list
            match($$)
        otherwise parse_error

procedure stmt_list
    case input_token of
        id, read, write : stmt; stmt_list
        $$ : skip      -- epsilon production
        otherwise parse_error

procedure stmt
    case input_token of
        id : match(id); match(:=); expr
        read : match(read); match(id)
        write : match(write); expr
        otherwise parse_error

procedure expr
    case input_token of
        id, number, ( : term; term_tail
        otherwise parse_error

procedure term_tail
    case input_token of
        +, - : add_op; term; term_tail
        ), id, read, write, $$ :
            skip      -- epsilon production
        otherwise parse_error

procedure term
    case input_token of
        id, number, ( : factor; factor_tail
        otherwise parse_error

```

Figure 2.16 Recursive descent parser for the calculator language. Execution begins in procedure `program`. The recursive calls trace out a traversal of the parse tree. Not shown is code to save this tree (or some similar structure) for use by later phases of the compiler. (*continued*)

```

procedure factor_tail
  case input_token of
    *, / : mult_op; factor; factor_tail
    +, -, ), id, read, write, $$ :
      skip      -- epsilon production
    otherwise parse_error

procedure factor
  case input_token of
    id : match(id)
    number : match(number)
    ( : match(() ; expr; match())
    otherwise parse_error

procedure add_op
  case input_token of
    + : match(+)
    - : match(-)
    otherwise parse_error

procedure mult_op
  case input_token of
    * : match(*)
    / : match(/)
    otherwise parse_error

```

Figure 2.16 (continued)

✓ CHECK YOUR UNDERSTANDING

20. What is the inherent “big-O” complexity of parsing? What is the complexity of parsers used in real compilers?
21. Summarize the difference between LL and LR parsing. Which one of them is also called “bottom-up”? “Top-down”? Which one is also called “predictive”? “Shift-reduce”? What do “LL” and “LR” stand for?
22. What kind of parser (top-down or bottom-up) is most common in production compilers?
23. Why are right-most derivations sometimes called *canonical*?
24. What is the significance of the “1” in LR(1)?
25. Why might we want (or need) different grammars for different parsing algorithms?
26. What is an *epsilon production*?
27. What are *recursive descent* parsers? Why are they used mostly for small languages?
28. How might a parser construct an explicit parse tree or syntax tree?



EXAMPLE 2.25

Driver and table for top-down parsing

Pseudocode for such a parser appears in Figure 2.18. The code is language independent. It requires a language-*dependent* parsing table, generally produced by an automatic tool. For the calculator grammar of Figure 2.15, the table appears in Figure 2.19.

EXAMPLE 2.26

Table-driven parse of the
“sum and average” program

```

terminal = 1 .. number_of_terminals
non_terminal = number_of_terminals + 1 .. number_of_symbols
symbol = 1 .. number_of_symbols
production = 1 .. number_of_productions

parse_tab : array [non_terminal, terminal] of record
    action : (predict, error)
    prod : production
prod_tab : array [production] of list of symbol
-- these two tables are created by a parser generator tool

parse_stack : stack of symbol

parse_stack.push(start_symbol)
loop
    expected_sym : symbol := parse_stack.pop
    if expected_sym ∈ terminal
        match(expected_sym)           -- as in Figure 2.16
        if expected_sym = $$ then return -- success!
    else
        if parse_tab[expected_sym, input_token].action = error
            parse_error
        else
            prediction : production := parse_tab[expected_sym, input_token].prod
            foreach sym : symbol in reverse prod_tab[prediction]
                parse_stack.push(sym)

```

Figure 2.18 Driver for a table-driven LL(1) parser.

Top-of-stack nonterminal	Current input token											
	id	number	read	write	:=	(	)	+	-	*	/	\$\$
<i>program</i>	1	—	1	1	—	—	—	—	—	—	—	1
<i>stmt_list</i>	2	—	2	2	—	—	—	—	—	—	—	3
<i>stmt</i>	4	—	5	6	—	—	—	—	—	—	—	—
<i>expr</i>	7	7	—	—	—	7	—	—	—	—	—	—
<i>term_tail</i>	9	—	9	9	—	—	9	8	8	—	—	9
<i>term</i>	10	10	—	—	—	10	—	—	—	—	—	—
<i>factor_tail</i>	12	—	12	12	—	—	12	12	12	11	11	12
<i>factor</i>	14	15	—	—	—	13	—	—	—	—	—	—
<i>add_op</i>	—	—	—	—	—	—	—	16	17	—	—	—
<i>mult_op</i>	—	—	—	—	—	—	—	—	—	18	19	—

Figure 2.19 LL(1) parse table for the calculator language. Table entries indicate the production to predict (as numbered in Figure 2.22). A dash indicates an error. When the top-of-stack symbol is a terminal, the appropriate action is always to match it against an incoming token from the scanner. An auxiliary table, not shown here, gives the right-hand-side symbols for each production.

Parse stack	Input stream	Comment
<i>program</i>	read A read B ...	initial stack contents
<i>stmt_list</i> \$\$	read A read B ...	predict <i>program</i> $\rightarrow$ <i>stmt_list</i> \$\$
<i>stmt stmt_list</i> \$\$	read A read B ...	predict <i>stmt_list</i> $\rightarrow$ <i>stmt stmt_list</i>
read id <i>stmt_list</i> \$\$	read A read B ...	predict <i>stmt</i> $\rightarrow$ read id
id <i>stmt_list</i> \$\$	A read B ...	match read
<i>stmt_list</i> \$\$	read B sum := ...	match id
<i>stmt stmt_list</i> \$\$	read B sum := ...	predict <i>stmt_list</i> $\rightarrow$ <i>stmt stmt_list</i>
read id <i>stmt_list</i> \$\$	read B sum := ...	predict <i>stmt</i> $\rightarrow$ read id
id <i>stmt_list</i> \$\$	B sum := ...	match read
<i>stmt_list</i> \$\$	sum := A + B ...	match id
<i>stmt stmt_list</i> \$\$	sum := A + B ...	predict <i>stmt_list</i> $\rightarrow$ <i>stmt stmt_list</i>
id := <i>expr stmt_list</i> \$\$	sum := A + B ...	predict <i>stmt</i> $\rightarrow$ id := <i>expr</i>
:= <i>expr stmt_list</i> \$\$	: = A + B ...	match id
<i>expr stmt_list</i> \$\$	A + B ...	match :=
<i>term term_tail stmt_list</i> \$\$	A + B ...	predict <i>expr</i> $\rightarrow$ <i>term term_tail</i>
factor <i>factor_tail term_tail stmt_list</i> \$\$	A + B ...	predict <i>term</i> $\rightarrow$ factor <i>factor_tail</i>
id <i>factor_tail term_tail stmt_list</i> \$\$	A + B ...	predict factor $\rightarrow$ id
<i>factor_tail term_tail stmt_list</i> \$\$	+ B write sum ...	match id
<i>term_tail stmt_list</i> \$\$	+ B write sum ...	predict <i>factor_tail</i> $\rightarrow$ ϵ
add_op <i>term term_tail stmt_list</i> \$\$	+ B write sum ...	predict <i>term_tail</i> $\rightarrow$ add_op <i>term term_tail</i>
+ <i>term term_tail stmt_list</i> \$\$	+ B write sum ...	predict add_op $\rightarrow$ +
<i>term term_tail stmt_list</i> \$\$	B write sum ...	match +
factor <i>factor_tail term_tail stmt_list</i> \$\$	B write sum ...	predict <i>term</i> $\rightarrow$ factor <i>factor_tail</i>
id <i>factor_tail term_tail stmt_list</i> \$\$	B write sum ...	predict factor $\rightarrow$ id
<i>factor_tail term_tail stmt_list</i> \$\$	write sum ...	match id
<i>term_tail stmt_list</i> \$\$	write sum write ...	predict <i>factor_tail</i> $\rightarrow$ ϵ
<i>stmt_list</i> \$\$	write sum write ...	predict <i>term_tail</i> $\rightarrow$ ϵ
<i>stmt stmt_list</i> \$\$	write sum write ...	predict <i>stmt_list</i> $\rightarrow$ <i>stmt stmt_list</i>
write <i>expr stmt_list</i> \$\$	write sum write ...	predict <i>stmt</i> $\rightarrow$ write <i>expr</i>
<i>expr stmt_list</i> \$\$	sum write sum / 2	match write
<i>term term_tail stmt_list</i> \$\$	sum write sum / 2	predict <i>expr</i> $\rightarrow$ <i>term term_tail</i>
factor <i>factor_tail term_tail stmt_list</i> \$\$	sum write sum / 2	predict <i>term</i> $\rightarrow$ factor <i>factor_tail</i>
id <i>factor_tail term_tail stmt_list</i> \$\$	sum write sum / 2	predict factor $\rightarrow$ id
<i>factor_tail term_tail stmt_list</i> \$\$	write sum / 2	match id
<i>term_tail stmt_list</i> \$\$	write sum / 2	predict <i>factor_tail</i> $\rightarrow$ ϵ
<i>stmt_list</i> \$\$	write sum / 2	predict <i>term_tail</i> $\rightarrow$ ϵ
<i>stmt stmt_list</i> \$\$	write sum / 2	predict <i>stmt_list</i> $\rightarrow$ <i>stmt stmt_list</i>
write <i>expr stmt_list</i> \$\$	write sum / 2	predict <i>stmt</i> $\rightarrow$ write <i>expr</i>
<i>expr stmt_list</i> \$\$	sum / 2	match write
<i>term term_tail stmt_list</i> \$\$	sum / 2	predict <i>expr</i> $\rightarrow$ <i>term term_tail</i>
factor <i>factor_tail term_tail stmt_list</i> \$\$	sum / 2	predict <i>term</i> $\rightarrow$ factor <i>factor_tail</i>
id <i>factor_tail term_tail stmt_list</i> \$\$	sum / 2	predict factor $\rightarrow$ id
<i>factor_tail term_tail stmt_list</i> \$\$	/ 2	match id
<i>mult_op factor factor_tail term_tail stmt_list</i> \$\$	/ 2	predict <i>factor_tail</i> $\rightarrow$ mult_op <i>factor factor_tail</i>
/ <i>factor factor_tail term_tail stmt_list</i> \$\$	/ 2	predict mult_op $\rightarrow$ /
<i>factor factor_tail term_tail stmt_list</i> \$\$	2	match /
number <i>factor_tail term_tail stmt_list</i> \$\$	2	predict factor $\rightarrow$ number
<i>factor_tail term_tail stmt_list</i> \$\$		match number
<i>term_tail stmt_list</i> \$\$		predict <i>factor_tail</i> $\rightarrow$ ϵ
<i>stmt_list</i> \$\$		predict <i>term_tail</i> $\rightarrow$ ϵ
\$\$		predict <i>stmt_list</i> $\rightarrow$ ϵ

Figure 2.20 Trace of a table-driven LL(1) parse of the sum-and-average program of Example 2.24.

the following actions. If the popped symbol is a terminal, the parser attempts to match it against an incoming token from the scanner. If the match fails, the parser announces a syntax error and initiates some sort of error recovery (see Section 2.3.4). If the popped symbol is a nonterminal, the parser uses that nonterminal together with the next available input token to index into a two-dimensional table that tells it which production to predict (or whether to announce a syntax error and initiate recovery).

Initially, the parse stack contains the start symbol of the grammar (in our case, *program*). When it predicts a production, the parser pushes the right-hand-side symbols onto the parse stack in reverse order, so the first of those symbols ends up at top-of-stack. The parse completes successfully when we match the end token, $\$$. Assuming that $\$$ appears only once in the grammar, at the end of the first production, and that the scanner returns this token only at end-of-file, any syntax error is guaranteed to manifest itself either as a failed match or as an error entry in the table. ■

Predict Sets

As we hinted at the end of Section 2.3.1, predict sets are defined in terms of simpler sets called FIRST and FOLLOW, where $\text{FIRST}(A)$ is the set of all tokens that could be the start of an A and $\text{FOLLOW}(A)$ is the set of all tokens that could come after an A in some valid program. If we extend the domain of FIRST in the obvious way to include *strings* of symbols, we then say that the predict set of a production $A \rightarrow \beta$ is $\text{FIRST}(\beta)$, plus $\text{FOLLOW}(A)$ if $\beta \Rightarrow^* \epsilon$. For notational convenience, we define the predicate EPS such that $\text{EPS}(\beta) \equiv \beta \Rightarrow^* \epsilon$.¹³

EXAMPLE 2.27

Predict sets for the calculator language

We can illustrate the algorithm to construct these sets using our calculator grammar (Figure 2.15). We begin with “obvious” facts about the grammar and build on them inductively. If we recast the grammar in plain BNF (no EBNF ‘|’ constructs), then it has 19 productions. The “obvious” facts arise from adjacent pairs of symbols in right-hand sides. In the first production, we can see that $\$ \in \text{FOLLOW}(\text{stmt_list})$. In the second ($\text{stmt_list} \rightarrow \epsilon$), $\text{EPS}(\text{stmt_list}) = \text{true}$. In the fourth production ($\text{stmt} \rightarrow \text{id} := \text{expr}$), $\text{id} \in \text{FIRST}(\text{stmt})$ and $:= \in \text{FOLLOW}(\text{id})$. In the fifth and sixth productions ($\text{stmt} \rightarrow \text{read id} \mid \text{write expr}$), $\{\text{read}, \text{write}\} \subset \text{FIRST}(\text{stmt})$, and $\text{id} \in \text{FOLLOW}(\text{read})$. The complete set of “obvious” facts appears in Figure 2.21.

From the “obvious” facts we can deduce a larger set of facts during a second pass over the grammar. For example, in the second production ($\text{stmt_list} \rightarrow \text{stmt stmt_list}$) we can deduce that $\{\text{id}, \text{read}, \text{write}\} \subset \text{FIRST}(\text{stmt_list})$, because we already know that $\{\text{id}, \text{read}, \text{write}\} \subset \text{FIRST}(\text{stmt})$, and a *stmt_list* can begin with

¹³ Following conventional notation, we use uppercase Roman letters near the beginning of the alphabet to represent nonterminals, uppercase Roman letters near the end of the alphabet to represent arbitrary grammar symbols (terminals or nonterminals), lowercase Roman letters near the beginning of the alphabet to represent terminals (tokens), lowercase Roman letters near the end of the alphabet to represent token strings, and lowercase Greek letters to represent strings of arbitrary symbols.

$program \rightarrow stmt_list \ \$\$$	$\$\$ \in FOLLOW(stmt_list)$
$stmt_list \rightarrow stmt \ stmt_list$	
$stmt_list \rightarrow \epsilon$	$EPS(stmt_list) = true$
$stmt \rightarrow id \ := \ expr$	$id \in FIRST(stmt) \text{ and } := \in FOLLOW(id)$
$stmt \rightarrow read \ id$	$read \in FIRST(stmt) \text{ and } id \in FOLLOW(read)$
$stmt \rightarrow write \ expr$	$write \in FIRST(stmt)$
$expr \rightarrow term \ term_tail$	
$term_tail \rightarrow add_op \ term \ term_tail$	
$term_tail \rightarrow \epsilon$	$EPS(term_tail) = true$
$term \rightarrow factor \ factor_tail$	
$factor_tail \rightarrow mult_op \ factor \ factor_tail$	
$factor_tail \rightarrow \epsilon$	$EPS(factor_tail) = true$
$factor \rightarrow (\ expr \)$	$(\in FIRST(factor) \text{ and }) \in FOLLOW(expr)$
$factor \rightarrow id$	$id \in FIRST(factor)$
$factor \rightarrow number$	$number \in FIRST(factor)$
$add_op \rightarrow +$	$+ \in FIRST(add_op)$
$add_op \rightarrow -$	$- \in FIRST(add_op)$
$mult_op \rightarrow *$	$* \in FIRST(mult_op)$
$mult_op \rightarrow /$	$/ \in FIRST(mult_op)$

Figure 2.21 “Obvious” facts about the LL(1) calculator grammar.

a *stmt*. Similarly, in the first production, we can deduce that $\$\$ \in FIRST(program)$, because we already know that $EPS(stmt_list) = true$.

In the eleventh production ($factor_tail \rightarrow mult_op \ factor \ factor_tail$), we can deduce that $\{ (, id, number \} \subset FOLLOW(mult_op)$, because we already know that $\{ (, id, number \} \subset FIRST(factor)$, and *factor* follows *mult_op* in the right-hand side. In the seventh production ($expr \rightarrow term \ term_tail$), we can deduce that $) \in FOLLOW(term_tail)$, because we already know that $) \in FOLLOW(expr)$, and a *term_tail* can be the last part of an *expr*. In this same production, we can also deduce that $) \in FOLLOW(term)$, because the *term_tail* can generate ϵ ($EPS(term_tail) = true$), allowing a *term* to be the last part of an *expr*.

There is more that we can learn from our second pass through the grammar, but the examples above cover all the different kinds of cases. To complete our calculation, we continue with additional passes over the grammar until we don’t learn any more (i.e., we don’t add anything to any of the FIRST and FOLLOW sets). We then construct the PREDICT sets. Final versions of all three sets appear in Figure 2.22. The parse table of Figure 2.19 follows directly from PREDICT. ■

The algorithm to compute EPS, FIRST, FOLLOW, and PREDICT sets appears, a bit more formally, in Figure 2.23. It relies on the following definitions.

$$\begin{aligned}
 EPS(\alpha) &\equiv \text{if } \alpha \Rightarrow^* \epsilon \text{ then true else false} \\
 FIRST(\alpha) &\equiv \{ c : \alpha \Rightarrow^* c \beta \} \\
 FOLLOW(A) &\equiv \{ c : S \Rightarrow^+ \alpha A c \beta \} \\
 PREDICT(A \rightarrow \alpha) &\equiv FIRST(\alpha) \cup (\text{if } EPS(\alpha) \text{ then } FOLLOW(A) \text{ else } \emptyset)
 \end{aligned}$$

FIRST

```

program {id, read, write, $$}
stmt_list {id, read, write}
stmt {id, read, write}
expr { (, id, number }
term_tail {+, -}
term { (, id, number }
factor_tail {*, /}
factor { (, id, number }
add_op {+, -}
mult_op {*, /}

```

Also note that $\text{FIRST}(c) = \{c\} \forall \text{ tokens } c$.

FOLLOW

```

id {+, -, *, /, ), :=, id, read, write, $$}
number {+, -, *, /, ), id, read, write, $$}
read {id}
write { (, id, number }
( { (, id, number }
) {+, -, *, /, ), id, read, write, $$}
:= { (, id, number }
+ { (, id, number }
- { (, id, number }
* { (, id, number }
/ { (, id, number }
$$  $\emptyset$ 
program  $\emptyset$ 
stmt_list {$$}
stmt {id, read, write, $$}

```

```

expr { ), id, read, write, $$}
term_tail { ), id, read, write, $$}
term {+, -, ), id, read, write, $$}
factor_tail {+, -, ), id, read, write, $$}
factor {+, -, *, /, ), id, read, write, $$}
add_op { (, id, number }
mult_op { (, id, number }

```

PREDICT

1. $\text{program} \rightarrow \text{stmt_list } \$\$ \{ \text{id, read, write, } \$\$ \}$
2. $\text{stmt_list} \rightarrow \text{stmt stmt_list } \{ \text{id, read, write} \}$
3. $\text{stmt_list} \rightarrow \epsilon \{ \$\$ \}$
4. $\text{stmt} \rightarrow \text{id} := \text{expr } \{ \text{id} \}$
5. $\text{stmt} \rightarrow \text{read id } \{ \text{read} \}$
6. $\text{stmt} \rightarrow \text{write expr } \{ \text{write} \}$
7. $\text{expr} \rightarrow \text{term term_tail } \{ (, \text{id, number} \}$
8. $\text{term_tail} \rightarrow \text{add_op term term_tail } \{ +, - \}$
9. $\text{term_tail} \rightarrow \epsilon \{), \text{id, read, write, } \$\$ \}$
10. $\text{term} \rightarrow \text{factor factor_tail } \{ (, \text{id, number} \}$
11. $\text{factor_tail} \rightarrow \text{mult_op factor factor_tail } \{ *, / \}$
12. $\text{factor_tail} \rightarrow \epsilon \{ +, -,), \text{id, read, write, } \$\$ \}$
13. $\text{factor} \rightarrow (\text{expr }) \{ (\}$
14. $\text{factor} \rightarrow \text{id } \{ \text{id} \}$
15. $\text{factor} \rightarrow \text{number } \{ \text{number} \}$
16. $\text{add_op} \rightarrow + \{ + \}$
17. $\text{add_op} \rightarrow - \{ - \}$
18. $\text{mult_op} \rightarrow * \{ * \}$
19. $\text{mult_op} \rightarrow / \{ / \}$

Figure 2.22 FIRST, FOLLOW, and PREDICT sets for the calculator language. $\text{EPS}(A)$ is true iff $A \in \{\text{stmt_list}, \text{term_tail}, \text{factor_tail}\}$.

Note that FIRST sets and EPS values for strings of length greater than one are calculated on demand; they are not stored explicitly. The algorithm is guaranteed to terminate (i.e., converge on a solution), because the sizes of the FIRST and FOLLOW sets are bounded by the number of terminals in the grammar.

If in the process of calculating PREDICT sets we find that some token belongs to the PREDICT set of more than one production with the same left-hand side, then the grammar is not LL(1), because we will not be able to choose which of the productions to employ when the left-hand side is at the top of the parse stack (or we are in the left-hand side's subroutine in a recursive descent parser) and we see the token coming up in the input. This sort of ambiguity is known as a *predict-predict conflict*; it can arise either because the same token can begin more than one right-hand side, or because it can begin one right-hand side and can also appear after the left-hand side in some valid program, and one possible right-hand side can generate ϵ .

```

-- EPS values and FIRST sets for all symbols:
  for all terminals  $c$ ,  $\text{EPS}(c) := \text{false}$ ;  $\text{FIRST}(c) := \{c\}$ 
  for all nonterminals  $X$ ,  $\text{EPS}(X) := \text{if } X \rightarrow \epsilon \text{ then true else false}$ ;  $\text{FIRST}(X) := \emptyset$ 
  repeat
    <outer> for all productions  $X \rightarrow Y_1 Y_2 \dots Y_k$ ,
      <inner> for  $i$  in  $1 \dots k$ 
        add  $\text{FIRST}(Y_i)$  to  $\text{FIRST}(X)$ 
        if not  $\text{EPS}(Y_i)$  (yet) then continue outer loop
       $\text{EPS}(X) := \text{true}$ 
  until no further progress

-- Subroutines for strings, similar to inner loop above:

 $\text{EPS}(X_1 X_2 \dots X_n)$ 
  for  $i$  in  $1 \dots n$ 
    if not  $\text{EPS}(X_i)$  then return false
  return true

 $\text{FIRST}(X_1 X_2 \dots X_n)$ 
   $\text{return\_value} := \emptyset$ 
  for  $i$  in  $1 \dots n$ 
    add  $\text{FIRST}(X_i)$  to  $\text{return\_value}$ 
    if not  $\text{EPS}(X_i)$  then return

-- FOLLOW sets for all symbols:
  for all symbols  $X$ ,  $\text{FOLLOW}(X) := \emptyset$ 
  repeat
    for all productions  $A \rightarrow \alpha B \beta$ ,
      add  $\text{FIRST}(\beta)$  to  $\text{FOLLOW}(B)$ 
    for all productions  $A \rightarrow \alpha B$ 
      or  $A \rightarrow \alpha B \beta$ , where  $\text{EPS}(\beta) = \text{true}$ ,
      add  $\text{FOLLOW}(A)$  to  $\text{FOLLOW}(B)$ 
  until no further progress

-- PREDICT sets for all productions:
  for all productions  $A \rightarrow \alpha$ 
     $\text{PREDICT}(A \rightarrow \alpha) := \text{FIRST}(\alpha) \cup (\text{if } \text{EPS}(\alpha) \text{ then } \text{FOLLOW}(A) \text{ else } \emptyset)$ 

```

Figure 2.23 Algorithm to calculate FIRST, FOLLOW, and PREDICT sets. The grammar is LL(1) if and only if the PREDICT sets are disjoint.

Writing an LL(1) Grammar

When working with a top-down parser generator, one has to acquire a certain facility in writing and modifying LL(1) grammars. The two most common obstacles to “LL(1)-ness” are *left recursion* and *common prefixes*.

EXAMPLE 2.28

Left recursion

A grammar is said to be left recursive if there is a nonterminal A such that $A \Rightarrow^+ A \alpha$ for some α . The trivial case occurs when the first symbol on the

right-hand side of a production is the same as the symbol on the left-hand side. Here again is the grammar from [Example 2.21](#), which cannot be parsed top-down:

$$\begin{aligned} id_list &\longrightarrow id_list_prefix ; \\ id_list_prefix &\longrightarrow id_list_prefix , id \\ &\longrightarrow id \end{aligned}$$

The problem is in the second and third productions; with *id_list_prefix* at top-of-stack and an *id* on the input, a predictive parser cannot tell which of the productions it should use. (Recall that left recursion is *desirable* in bottom-up grammars, because it allows recursive constructs to be discovered incrementally, as in [Figure 2.14](#).) ■

EXAMPLE 2.29

Common prefixes

Common prefixes occur when two different productions with the same left-hand side begin with the same symbol or symbols. Here is an example that commonly appears in languages descended from Algol:

$$\begin{aligned} stmt &\longrightarrow id := expr \\ &\longrightarrow id (argument_list) \qquad \text{-- procedure call} \end{aligned}$$

Clearly *id* is in the FIRST set of both right-hand sides, and therefore in the PREDICT set of both productions. ■

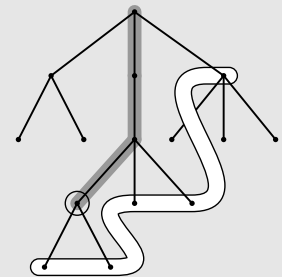
Both left recursion and common prefixes can be removed from a grammar mechanically. The general case is a little tricky ([Exercise 2.22](#)), because the prediction problem may be an indirect one (e.g., $S \longrightarrow A \alpha$ and $A \longrightarrow S \beta$, or

DESIGN & IMPLEMENTATION

Recursive descent and table-driven LL parsing

When trying to understand the connection between recursive descent and table-driven LL parsing, it is tempting to imagine that the explicit stack of the table-driven parser mirrors the implicit call stack of the recursive descent parser, but this is not the case.

A better way to visualize the two implementations of top-down parsing is to remember that both are discovering a parse tree via depth-first left-to-right traversal. When we are at a given point in the parse—say the circled node in the tree shown here—the implicit call stack of a recursive descent parser holds a frame for each of the nodes on the path back to the root, created when the routine corresponding to that node was called. (This path is shown in grey.)



But these nodes are immaterial. What matters for the rest of the parse—as shown on the white path here—are the *upcoming calls* on the case statement arms of the recursive descent routines. Those calls—those parse tree nodes—are precisely the contents of the explicit stack of a table-driven LL parser.

$S \rightarrow A \alpha$, $S \rightarrow B \beta$, $A \Rightarrow^* c \gamma$, and $B \Rightarrow^* c \delta$). We can see the general idea in the examples above, however.

EXAMPLE 2.30

Eliminating left recursion

Our left-recursive definition of *id_list* can be replaced by the right-recursive variant we saw in [Example 2.20](#):

```
id_list → id id_list_tail
id_list_tail → , id id_list_tail
id_list_tail → ;
```

EXAMPLE 2.31

Left factoring

Our common-prefix definition of *stmt* can be made LL(1) by a technique called *left factoring*:

```
stmt → id stmt_list_tail
stmt_list_tail → := expr | ( argument_list )
```

Of course, simply eliminating left recursion and common prefixes is *not* guaranteed to make a grammar LL(1). There are infinitely many non-LL *languages*—languages for which no LL grammar exists—and the mechanical transformations to eliminate left recursion and common prefixes work on their grammars just fine. Fortunately, the few non-LL languages that arise in practice can generally be handled by augmenting the parsing algorithm with one or two simple heuristics.

EXAMPLE 2.32

Parsing a “dangling else”

The best known example of a “not quite LL” construct arises in languages like Pascal, in which the *else* part of an *if* statement is optional. The natural grammar fragment

```
stmt → if condition then_clause else_clause | other_stmt
then_clause → then stmt
else_clause → else stmt | ε
```

is ambiguous (and thus neither LL nor LR); it allows the *else* in *if* C_1 *then* *if* C_2 *then* S_1 *else* S_2 to be paired with either *then*. The less natural grammar fragment

```
stmt → balanced_stmt | unbalanced_stmt
balanced_stmt → if condition then balanced_stmt else balanced_stmt
                | other_stmt
unbalanced_stmt → if condition then stmt
                | if condition then balanced_stmt else unbalanced_stmt
```

can be parsed bottom-up but not top-down (there is *no* pure top-down grammar for Pascal *else* statements). A *balanced_stmt* is one with the same number of *thens* and *elses*. An *unbalanced_stmt* has more *thens*.

The usual approach, whether parsing top-down or bottom-up, is to use the ambiguous grammar together with a “disambiguating rule,” which says that in the case of a conflict between two possible productions, the one to use is the one that occurs first, textually, in the grammar. In the ambiguous fragment above, the fact that *else_clause* $\rightarrow$ *else stmt* comes before *else_clause* $\rightarrow$ ϵ ends up pairing the *else* with the nearest *then*, as desired.

EXAMPLE 2.33

“Dangling else” program bug

Better yet, a language designer can avoid this sort of problem by choosing different syntax. The ambiguity of the *dangling else* problem in Pascal leads to problems not only in parsing, but in writing and maintaining correct programs. Most Pascal programmers have at one time or another written a program like this one:

```
if P <> nil then
  if P^.val = goal then
    foundIt := true
  else
    endOfList := true
```

Indentation notwithstanding, the Pascal manual states that an `else` clause matches the closest unmatched `then`—in this case the inner one—which is clearly not what the programmer intended. To get the desired effect, the Pascal programmer must write

```
if P <> nil then begin
  if P^.val = goal then
    foundIt := true
  end
else
  endOfList := true
```

Many other Algol-family languages (including Modula, Modula-2, and Oberon, all more recent inventions of Pascal’s designer, Niklaus Wirth) require explicit *end markers* on all structured statements. The grammar fragment for `if` statements in Modula-2 looks something like this:

```
stmt → IF condition then_clause else_clause END | other_stmt
then_clause → THEN stmt_list
else_clause → ELSE stmt_list | ε
```

The addition of the `END` eliminates the ambiguity. ■

Modula-2 uses `END` to terminate all its structured statements. Ada and Fortran 77 end an `if` with `end if` (and a `while` with `end while`, etc.). Algol 68 creates its terminators by spelling the initial keyword backward (`if... fi`, `case... esac`, `do... od`, etc.).

One problem with end markers is that they tend to bunch up. In Pascal one can write

EXAMPLE 2.34

End markers for structured statements

EXAMPLE 2.35

The need for `elsif`

DESIGN & IMPLEMENTATION**The dangling else**

A simple change in language syntax—eliminating the *dangling else*—not only reduces the chance of programming errors, but also significantly simplifies parsing. For more on the *dangling else* problem, see [Exercise 2.27](#) and [Section 6.4](#).

```

if A = B then ...
else if A = C then ...
else if A = D then ...
else if A = E then ...
else ...

```

With end markers this becomes

```

if A = B then ...
else if A = C then ...
else if A = D then ...
else if A = E then ...
else ...
end end end end

```

To avoid this awkwardness, languages with end markers generally provide an `elsif` keyword (sometimes spelled `elif`):

```

if A = B then ...
elsif A = C then ...
elsif A = D then ...
elsif A = E then ...
else ...
end

```

With `elsif` clauses added, the Modula-2 grammar fragment for `if` statements looks like this:

```

stmt  → IF condition then_clause elsif_clauses else_clause END | other_stmt
then_clause → THEN stmt_list
elsif_clauses → ELSIF condition then_clause elsif_clauses | ε
else_clause → ELSE stmt_list | ε

```

✓ CHECK YOUR UNDERSTANDING

29. Discuss the similarities and differences between recursive descent and table-driven top-down parsing.
30. What are FIRST and FOLLOW sets? What are they used for?
31. Under what circumstances does a top-down parser predict the production $A \rightarrow \alpha$?
32. What sorts of “obvious” facts form the basis of FIRST set and FOLLOW set construction?
33. Outline the algorithm used to complete the construction of FIRST and FOLLOW sets. How do we know when we are done?

34. How do we know when a grammar is not LL(1)?
35. Describe two common idioms in context-free grammars that cannot be parsed top-down.
36. What is the “dangling else” problem? How is it avoided in modern languages?

2.3.3 Bottom-Up Parsing

Conceptually, as we saw at the beginning of [Section 2.3](#), a bottom-up parser works by maintaining a forest of partially completed subtrees of the parse tree, which it joins together whenever it recognizes the symbols on the right-hand side of some production used in the right-most derivation of the input string. It creates a new internal node and makes the roots of the joined-together trees the children of that node.

In practice, a bottom-up parser is almost always table-driven. It keeps the roots of its partially completed subtrees on a stack. When it accepts a new token from the scanner, it *shifts* the token into the stack. When it recognizes that the top few symbols on the stack constitute a right-hand side, it *reduces* those symbols to their left-hand side by popping them off the stack and pushing the left-hand side in their place. The role of the stack is the first important difference between top-down and bottom-up parsing: a top-down parser’s stack contains a list of what the parser expects to see in the future; a bottom-up parser’s stack contains a record of what the parser has already seen in the past.

Canonical Derivations

We also noted earlier that the actions of a bottom-up parser trace out a right-most (canonical) derivation in reverse. The roots of the partial subtrees, left-to-right, together with the remaining input, constitute a sentential form of the right-most derivation. On the right-hand side of [Figure 2.13](#), for example, we have the following series of steps.

EXAMPLE 2.36

Derivation of an id list

Stack contents (roots of partial trees)	Remaining input
ε	A, B, C;
id (A)	, B, C;
id (A) ,	B, C;
id (A) , id (B)	, C;
id (A) , id (B) ,	C;
id (A) , id (B) , id (C)	;
id (A) , id (B) , id (C) ;	
id (A) , id (B) , <u>id (C) id_list_tail</u>	
id (A) , <u>id (B) id_list_tail</u>	
<u>id (A) id_list_tail</u>	
<u>id_list</u>	

1. $program \rightarrow stmt_list \$\$$
2. $stmt_list \rightarrow stmt_list stmt$
3. $stmt_list \rightarrow stmt$
4. $stmt \rightarrow id := expr$
5. $stmt \rightarrow read id$
6. $stmt \rightarrow write expr$
7. $expr \rightarrow term$
8. $expr \rightarrow expr add_op term$
9. $term \rightarrow factor$
10. $term \rightarrow term mult_op factor$
11. $factor \rightarrow (expr)$
12. $factor \rightarrow id$
13. $factor \rightarrow number$
14. $add_op \rightarrow +$
15. $add_op \rightarrow -$
16. $mult_op \rightarrow *$
17. $mult_op \rightarrow /$

Figure 2.24 LR(1) grammar for the calculator language. Productions have been numbered for reference in future figures.

The last four lines (the ones that don't just shift tokens into the forest) correspond to the right-most derivation:

$$\begin{aligned}
 id_list &\Rightarrow id \ id_list_tail \\
 &\Rightarrow id \ , \ id \ id_list_tail \\
 &\Rightarrow id \ , \ id \ , \ id \ id_list_tail \\
 &\Rightarrow id \ , \ id \ , \ id \ ;
 \end{aligned}$$

The symbols that need to be joined together at each step of the parse to represent the next step of the backward derivation are called the *handle* of the sentential form. In the parse trace above, the handles are underlined. ■

EXAMPLE 2.37

Bottom-up grammar for the calculator language

In our *id_list* example, no handles were found until the entire input had been shifted onto the stack. In general this will not be the case. We can obtain a more realistic example by examining an LR version of our calculator language, shown in Figure 2.24. While the LL grammar of Figure 2.15 can be parsed bottom-up, the version in Figure 2.24 is preferable for two reasons. First, it uses a left-recursive production for *stmt_list*. Left recursion allows the parser to collapse long statement lists as it goes along, rather than waiting until the entire list is on the stack and then collapsing it from the end. Second, it uses left-recursive productions for *expr* and *term*. These productions capture left associativity while still keeping an operator and its operands together in the same right-hand side, something we were unable to do in a top-down grammar. ■

EXAMPLE 2.38

Bottom-up parse of the
“sum and average” program

Modeling a Parse with LR Items

Suppose we are to parse the sum-and-average program from [Example 2.24](#):

```
read A
read B
sum := A + B
write sum
write sum / 2
```

The key to success will be to figure out when we have reached the end of a right-hand side—that is, when we have a handle at the top of the parse stack. The trick is to keep track of the set of productions we might be “in the middle of” at any particular time, together with an indication of where in those productions we might be.

When we begin execution, the parse stack is empty and we are at the beginning of the production for *program*. (In general, we can assume that there is only one production with the start symbol on the left-hand side; it is easy to modify any grammar to make this the case.) We can represent our location—more specifically, the location represented by the top of the parse stack—with a $\bullet$ in the right-hand side of the production:

$$\text{program} \longrightarrow \bullet \text{ stmt_list } \$\$$$

When augmented with a $\bullet$, a production is called an LR *item*. Since the $\bullet$ in this item is immediately in front of a nonterminal—namely *stmt_list*—we may be about to see the yield of that nonterminal coming up on the input. This possibility implies that we may be at the beginning of some production with *stmt_list* on the left-hand side:

$$\begin{aligned} \text{program} &\longrightarrow \bullet \text{ stmt_list } \$\$ \\ \text{stmt_list} &\longrightarrow \bullet \text{ stmt_list } \text{stmt} \\ \text{stmt_list} &\longrightarrow \bullet \text{ stmt} \end{aligned}$$

And, since *stmt* is a nonterminal, we may also be at the beginning of any production whose left-hand side is *stmt*:

$$\begin{aligned} \text{program} &\longrightarrow \bullet \text{ stmt_list } \$\$ && \text{(State 0)} \\ \text{stmt_list} &\longrightarrow \bullet \text{ stmt_list } \text{stmt} \\ \text{stmt_list} &\longrightarrow \bullet \text{ stmt} \\ \text{stmt} &\longrightarrow \bullet \text{ id } := \text{expr} \\ \text{stmt} &\longrightarrow \bullet \text{ read id} \\ \text{stmt} &\longrightarrow \bullet \text{ write expr} \end{aligned}$$

Since all of these last productions begin with a terminal, no additional items need to be added to our list. The original item ($\text{program} \longrightarrow \bullet \text{ stmt_list } \$\$$) is called the *basis* of the list. The additional items are its *closure*. The list represents the

initial state of the parser. As we shift and reduce, the set of items will change, always indicating which productions *may* be the right one to use next in the derivation of the input string. If we reach a state in which some item has the $\bullet$ at the end of the right-hand side, we can reduce by that production. Otherwise, as in the current situation, we must shift. Note that if we need to shift, but the incoming token cannot follow the $\bullet$ in any item of the current state, then a syntax error has occurred. We will consider error recovery in more detail in Section ©2.3.4.

Our upcoming token is a *read*. Once we shift it onto the stack, we know we are in the following state:

$$stmt \longrightarrow read \bullet id \quad (\text{State 1})$$

This state has a single basis item and an empty closure—the $\bullet$ precedes a terminal. After shifting the *A*, we have

$$stmt \longrightarrow read id \bullet \quad (\text{State 1'})$$

We now know that *read id* is the handle, and we must reduce. The reduction pops two symbols off the parse stack and pushes a *stmt* in their place, but what should the new state be? We can see the answer if we imagine moving back in time to the point at which we shifted the *read*—the first symbol of the right-hand side. At that time we were in the state labeled “State 0” above, and the upcoming tokens on the input (though we didn’t look at them at the time) were *read id*. We have now consumed these tokens, and we know that they constituted a *stmt*. By pushing a *stmt* onto the stack, we have in essence replaced *read id* with *stmt* on the input stream, and have then “shifted” the nonterminal, rather than its yield, into the stack. Since one of the items in State 0 was

$$stmt_list \longrightarrow \bullet stmt$$

we now have

$$stmt_list \longrightarrow stmt \bullet \quad (\text{State 0'})$$

Again we must reduce. We remove the *stmt* from the stack and push a *stmt_list* in its place. Again we can see this as “shifting” a *stmt_list* when in State 0. Since two of the items in State 0 have a *stmt_list* after the $\bullet$, we don’t know (without looking ahead) which of the productions will be the next to be used in the derivation, but we don’t have to know. The key advantage of bottom-up parsing over top-down parsing is that we don’t need to predict ahead of time which production we shall be expanding.

Our new state is as follows:

$$program \longrightarrow stmt_list \bullet \$\$ \quad (\text{State 2})$$

$$stmt_list \longrightarrow stmt_list \bullet stmt$$

$$stmt \longrightarrow \bullet id := expr$$

$$stmt \longrightarrow \bullet read id$$

$$stmt \longrightarrow \bullet write expr$$

The first two productions are the basis; the others are the closure. Since no item has a $\bullet$ at the end, we shift the next token, which happens again to be a read, taking us back to State 1. Shifting the B takes us to State 1' again, at which point we reduce. This time however, we go back to State 2 rather than State 0 before shifting the left-hand-side *stmt*. Why? Because we were in State 2 when we began to read the right-hand-side. ■

The Characteristic Finite State Machine and LR Parsing Variants

An LR-family parser keeps track of the states it has traversed by pushing them into the parse stack, along with the grammar symbols. It is in fact the states (rather than the symbols) that drive the parsing algorithm: they tell us what state we were in at the beginning of a right-hand side. Specifically, when the combination of state and input tells us we need to reduce using production $A \rightarrow \alpha$, we pop $length(\alpha)$ symbols off the stack, together with the record of states we moved through while shifting those symbols. These pops expose the state we were in immediately prior to the shifts, allowing us to return to that state and proceed as if we had seen A in the first place.

We can think of the shift rules of an LR-family parser as the transition function of a finite automaton, much like the automata we used to model scanners. Each state of the automaton corresponds to a list of items that indicate where the parser might be at some specific point in the parse. The transition for input symbol X (which may be either a terminal or a nonterminal) moves to a state whose basis consists of items in which the $\bullet$ has been moved across an X in the right-hand side, plus whatever items need to be added as closure. The lists are constructed by a bottom-up parser generator in order to build the automaton, but are not needed during parsing.

It turns out that the simpler members of the LR family of parsers—LR(0), SLR(1), and LALR(1)—all use the same automaton, called the *characteristic finite-state machine*, or CFSM. Full LR parsers use a machine with (for most grammars) a much larger number of states. The differences between the algorithms lie in how they deal with states that contain a *shift-reduce conflict*—one item with the $\bullet$ in front of a terminal (suggesting the need for a shift) and another with the $\bullet$ at the end of the right-hand side (suggesting the need for a reduction). An LR(0) parser works only when there are no such states. It can be proven that with the addition of an end-marker (i.e., $\$$), any language that can be deterministically parsed bottom-up has an LR(0) grammar. Unfortunately, the LR(0) grammars for real programming languages tend to be prohibitively large and unintuitive.

SLR (simple LR) parsers peek at upcoming input and use FOLLOW sets to resolve conflicts. An SLR parser will call for a reduction via $A \rightarrow \alpha$ only if the upcoming token(s) are in $FOLLOW(\alpha)$. It will still see a conflict, however, if the tokens are also in the FIRSTset of any of the symbols that follow a $\bullet$ in other items of the state. As it turns out, there are important cases in which a token may follow a given nonterminal somewhere in a valid program, but never in a context described by the current state. For these cases global FOLLOW sets are too crude.

LALR (look-ahead LR) parsers improve on SLR by using *local* (state-specific) look-ahead instead.

Conflicts can still arise in an LALR parser when the same set of items can occur on two different paths through the CFSM. Both paths will end up in the same state, at which point state-specific look-ahead can no longer distinguish between them. A full LR parser duplicates states in order to keep paths disjoint when their local look-aheads are different.

LALR parsers are the most common bottom-up parsers in practice. They are the same size and speed as SLR parsers, but are able to resolve more conflicts. Full LR parsers for real programming languages tend to be very large. Several researchers have developed techniques to reduce the size of full-LR tables, but LALR works sufficiently well in practice that the extra complexity of full LR is usually not required. Yacc/bison produces C code for an LALR parser.

Bottom-Up Parsing Tables

Like a table-driven LL(1) parser, an SLR(1), LALR(1), or LR(1) parser executes a loop in which it repeatedly inspects a two-dimensional table to find out what action to take. However, instead of using the current input token and top-of-stack nonterminal to index into the table, an LR-family parser uses the current input token and the current parser state (which can be found at the top of the stack). “Shift” table entries indicate the state that should be pushed. “Reduce” table entries indicate the number of states that should be popped and the nonterminal that should be pushed back onto the input stream, to be shifted by the state uncovered by the pops. There is always one popped state for every symbol on the right-hand side of the reducing production. The state to be pushed next can be found by indexing into the table using the uncovered state and the newly recognized nonterminal.

EXAMPLE 2.39

CFSM for the bottom-up calculator grammar

The CFSM for our bottom-up version of the calculator grammar appears in Figure 2.25. States 6, 7, 9, and 13 contain potential shift-reduce conflicts, but all of these can be resolved with global FOLLOW sets. SLR parsing therefore suffices. In State 6, for example, $\text{FIRST}(\text{add_op}) \cap \text{FOLLOW}(\text{stmt}) = \emptyset$. In addition to shift and reduce rules, we allow the parse table as an optimization to contain rules of the form “shift and then reduce.” This optimization serves to eliminate trivial states such as $1'$ and $0'$ in Example 2.38, which had only a single item, with the $\bullet$ at the end.

A pictorial representation of the CFSM appears in Figure 2.26. A tabular representation, suitable for use in a table-driven parser, appears in Figure 2.27. Pseudocode for the (language-independent) parser driver appears in Figure 2.28. A trace of the parser’s actions on the sum-and-average program appears in Figure 2.29. ■

Handling Epsilon Productions

EXAMPLE 2.40

Epsilon productions in the bottom-up calculator grammar

The careful reader may have noticed that the grammar of Figure 2.24, in addition to using left-recursive rules for *stmt_list*, *expr*, and *term*, differs from the grammar of Figure 2.15 in one other way: it defines a *stmt_list* to be a sequence of one or more *stmts*, rather than zero or more. (This means, of course, that it defines a

different language.) To capture the same language as Figure 2.15, production 3 in Figure 2.24,

$$stmt_list \longrightarrow stmt$$

would need to be replaced with

$$stmt_list \longrightarrow \epsilon$$

Note that it does in general make sense to have an empty statement list. In the calculator language it simply permits an empty program, which is admittedly silly. In real languages, however, it allows the body of a structured statement to be empty, which can be very useful. One frequently wants one arm of a case or multiway if... then... else statement to be empty, and an empty while loop allows a parallel program (or the operating system) to wait for a signal from another process or an I/O device.

EXAMPLE 2.41

CFSM with epsilon productions

If we look at the CFSM for the calculator language, we discover that State 0 is the only state that needs to be changed in order to allow empty statement lists. The item

$$stmt_list \longrightarrow \bullet stmt$$

becomes

$$stmt_list \longrightarrow \bullet \epsilon$$

which is equivalent to

$$stmt_list \longrightarrow \epsilon \bullet$$

or simply

$$stmt_list \longrightarrow \bullet$$

The entire state is then

$$\begin{array}{ll} \underline{program \longrightarrow \bullet stmt_list \ \$\$} & \text{on } stmt_list \text{ shift and goto 2} \\ stmt_list \longrightarrow \bullet stmt_list \ stmt & \\ stmt_list \longrightarrow \bullet & \text{on } \$\$ \text{ reduce (pop 0 states, push } stmt_list \text{ on input)} \\ stmt \longrightarrow \bullet id := expr & \text{on } id \text{ shift and goto 3} \\ stmt \longrightarrow \bullet read \ id & \text{on } read \text{ shift and goto 1} \\ stmt \longrightarrow \bullet write \ expr & \text{on } write \text{ shift and goto 4} \end{array}$$

The look-ahead for item

$$stmt_list \longrightarrow \bullet$$

is FOLLOW(*stmt_list*), which is the end-marker, $\$ \$$. Since $\$ \$$ does not appear in the look-aheads for any other item in this state, our grammar is still SLR(1). It is worth noting that epsilon productions commonly prevent a grammar from being LR(0): if such a production shares a state with an item in which the dot precedes a terminal, we won't be able to tell whether to "recognize" ϵ without peeking ahead.

State	Transitions
0. <u>$program \rightarrow \bullet stmt_list \\$\\$</u> $stmt_list \rightarrow \bullet stmt_list stmt$ $stmt_list \rightarrow \bullet stmt$ $stmt \rightarrow \bullet id := expr$ $stmt \rightarrow \bullet read id$ $stmt \rightarrow \bullet write expr$	on $stmt_list$ shift and goto 2 on $stmt$ shift and reduce (pop 1 state, push $stmt_list$ on input) on id shift and goto 3 on $read$ shift and goto 1 on $write$ shift and goto 4
1. $stmt \rightarrow read \bullet id$	on id shift and reduce (pop 2 states, push $stmt$ on input)
2. <u>$program \rightarrow stmt_list \bullet \\$\\$</u> <u>$stmt_list \rightarrow stmt_list \bullet stmt$</u> $stmt \rightarrow \bullet id := expr$ $stmt \rightarrow \bullet read id$ $stmt \rightarrow \bullet write expr$	on $\$ \$$ shift and reduce (pop 2 states, push $program$ on input) on $stmt$ shift and reduce (pop 2 states, push $stmt_list$ on input) on id shift and goto 3 on $read$ shift and goto 1 on $write$ shift and goto 4
3. $stmt \rightarrow id \bullet := expr$	on $:=$ shift and goto 5
4. <u>$stmt \rightarrow write \bullet expr$</u> $expr \rightarrow \bullet term$ $expr \rightarrow \bullet expr add_op term$ $term \rightarrow \bullet factor$ $term \rightarrow \bullet term mult_op factor$ $factor \rightarrow \bullet (expr)$ $factor \rightarrow \bullet id$ $factor \rightarrow \bullet number$	on $expr$ shift and goto 6 on $term$ shift and goto 7 on $factor$ shift and reduce (pop 1 state, push $term$ on input) on $($ shift and goto 8 on id shift and reduce (pop 1 state, push $factor$ on input) on $number$ shift and reduce (pop 1 state, push $factor$ on input)
5. <u>$stmt \rightarrow id := \bullet expr$</u> $expr \rightarrow \bullet term$ $expr \rightarrow \bullet expr add_op term$ $term \rightarrow \bullet factor$ $term \rightarrow \bullet term mult_op factor$ $factor \rightarrow \bullet (expr)$ $factor \rightarrow \bullet id$ $factor \rightarrow \bullet number$	on $expr$ shift and goto 9 on $term$ shift and goto 7 on $factor$ shift and reduce (pop 1 state, push $term$ on input) on $($ shift and goto 8 on id shift and reduce (pop 1 state, push $factor$ on input) on $number$ shift and reduce (pop 1 state, push $factor$ on input)
6. <u>$stmt \rightarrow write expr \bullet$</u> <u>$expr \rightarrow expr \bullet add_op term$</u> $add_op \rightarrow \bullet +$ $add_op \rightarrow \bullet -$	on FOLLOW($stmt$) = { $id, read, write, \$ \$$ } reduce (pop 2 states, push $stmt$ on input) on add_op shift and goto 10 on $+$ shift and reduce (pop 1 state, push add_op on input) on $-$ shift and reduce (pop 1 state, push add_op on input)

Figure 2.25 CFSM for the calculator grammar (Figure 2.24). Basis and closure items in each state are separated by a horizontal rule. Trivial reduce-only states have been eliminated by use of “shift and reduce” transitions. (continued)

State	Transitions
7. $expr \rightarrow term \bullet$ $term \rightarrow term \bullet mult_op factor$ $mult_op \rightarrow \bullet *$ $mult_op \rightarrow \bullet /$	on FOLLOW($expr$) = {id, read, write, \$\$, }, +, -} reduce (pop 1 state, push $expr$ on input) on $mult_op$ shift and goto 11 on $*$ shift and reduce (pop 1 state, push $mult_op$ on input) on $/$ shift and reduce (pop 1 state, push $mult_op$ on input)
8. $factor \rightarrow (\bullet expr)$ $expr \rightarrow \bullet term$ $expr \rightarrow \bullet expr add_op term$ $term \rightarrow \bullet factor$ $term \rightarrow \bullet term mult_op factor$ $factor \rightarrow \bullet (expr)$ $factor \rightarrow \bullet id$ $factor \rightarrow \bullet number$	on $expr$ shift and goto 12 on $term$ shift and goto 7 on $factor$ shift and reduce (pop 1 state, push $term$ on input) on (shift and goto 8 on id shift and reduce (pop 1 state, push $factor$ on input) on number shift and reduce (pop 1 state, push $factor$ on input)
9. $stmt \rightarrow id := expr \bullet$ $expr \rightarrow expr \bullet add_op term$ $add_op \rightarrow \bullet +$ $add_op \rightarrow \bullet -$	on FOLLOW($stmt$) = {id, read, write, \$\$} reduce (pop 3 states, push $stmt$ on input) on add_op shift and goto 10 on + shift and reduce (pop 1 state, push add_op on input) on - shift and reduce (pop 1 state, push add_op on input)
10. $expr \rightarrow expr add_op \bullet term$ $term \rightarrow \bullet factor$ $term \rightarrow \bullet term mult_op factor$ $factor \rightarrow \bullet (expr)$ $factor \rightarrow \bullet id$ $factor \rightarrow \bullet number$	on $term$ shift and goto 13 on $factor$ shift and reduce (pop 1 state, push $term$ on input) on (shift and goto 8 on id shift and reduce (pop 1 state, push $factor$ on input) on number shift and reduce (pop 1 state, push $factor$ on input)
11. $term \rightarrow term mult_op \bullet factor$ $factor \rightarrow \bullet (expr)$ $factor \rightarrow \bullet id$ $factor \rightarrow \bullet number$	on $factor$ shift and reduce (pop 3 states, push $term$ on input) on (shift and goto 8 on id shift and reduce (pop 1 state, push $factor$ on input) on number shift and reduce (pop 1 state, push $factor$ on input)
12. $factor \rightarrow (expr \bullet)$ $expr \rightarrow expr \bullet add_op term$ $add_op \rightarrow \bullet +$ $add_op \rightarrow \bullet -$	on) shift and reduce (pop 3 states, push $factor$ on input) on add_op shift and goto 10 on + shift and reduce (pop 1 state, push add_op on input) on - shift and reduce (pop 1 state, push add_op on input)
13. $expr \rightarrow expr add_op term \bullet$ $term \rightarrow term \bullet mult_op factor$ $mult_op \rightarrow \bullet *$ $mult_op \rightarrow \bullet /$	on FOLLOW($expr$) = {id, read, write, \$\$, }, +, -} reduce (pop 3 states, push $expr$ on input) on $mult_op$ shift and goto 11 on $*$ shift and reduce (pop 1 state, push $mult_op$ on input) on $/$ shift and reduce (pop 1 state, push $mult_op$ on input)

Figure 2.25 (continued)

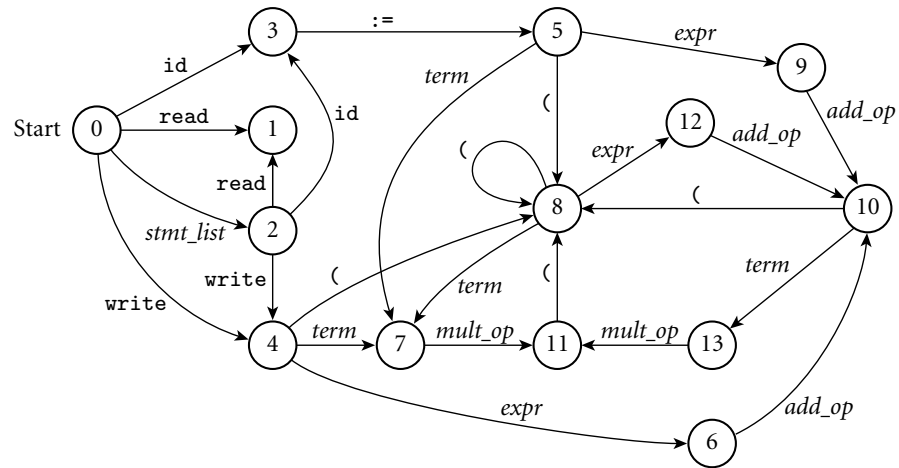


Figure 2.26 Pictorial representation of the CFSM of Figure 2.25. Reduce actions are not shown.

Top-of-stack state	Current input symbol																		
	sl	s	e	t	f	ao	mo	id	lit	r	w	:=	(	)	+	-	*	/	\$\$
0	s2	b3	-	-	-	-	-	s3	-	s1	s4	-	-	-	-	-	-	-	-
1	-	-	-	-	-	-	-	b5	-	-	-	-	-	-	-	-	-	-	-
2	-	b2	-	-	-	-	-	s3	-	s1	s4	-	-	-	-	-	-	-	b1
3	-	-	-	-	-	-	-	-	-	-	-	s5	-	-	-	-	-	-	-
4	-	-	s6	s7	b9	-	-	b12	b13	-	-	-	s8	-	-	-	-	-	-
5	-	-	s9	s7	b9	-	-	b12	b13	-	-	-	s8	-	-	-	-	-	-
6	-	-	-	-	-	s10	-	r6	-	r6	r6	-	-	-	b14	b15	-	-	r6
7	-	-	-	-	-	-	s11	r7	-	r7	r7	-	-	r7	r7	r7	b16	b17	r7
8	-	-	s12	s7	b9	-	-	b12	b13	-	-	-	s8	-	-	-	-	-	-
9	-	-	-	-	-	s10	-	r4	-	r4	r4	-	-	-	b14	b15	-	-	r4
10	-	-	-	s13	b9	-	-	b12	b13	-	-	-	s8	-	-	-	-	-	-
11	-	-	-	-	b10	-	-	b12	b13	-	-	-	s8	-	-	-	-	-	-
12	-	-	-	-	-	s10	-	-	-	-	-	-	-	b11	b14	b15	-	-	-
13	-	-	-	-	-	-	s11	r8	-	r8	r8	-	-	r8	r8	r8	b16	b17	r8

Figure 2.27 SLR(1) parse table for the calculator language. Table entries indicate whether to shift (s), reduce (r), or shift and then reduce (b). The accompanying number is the new state when shifting, or the production that has been recognized when (shifting and) reducing. Production numbers are given in Figure 2.24. Symbol names have been abbreviated for the sake of formatting. A dash indicates an error. An auxiliary table, not shown here, gives the left-hand-side symbol and right-hand-side length for each production.

✓ CHECK YOUR UNDERSTANDING

37. What is the *handle* of a right sentential form?
38. Explain the significance of the characteristic finite-state machine in LR parsing.


```

state = 1 .. number_of_states
symbol = 1 .. number_of_symbols
production = 1 .. number_of_productions
action_rec = record
    action : (shift, reduce, shift_reduce, error)
    new_state : state
    prod : production

parse_tab : array [symbol, state] of action_rec
prod_tab : array [production] of record
    lhs : symbol
    rhs_len : integer
-- these two tables are created by a parser generator tool

parse_stack : stack of record
    sym : symbol
    st : state

parse_stack.push((null, start_state))
cur_sym : symbol := scan -- get new token from scanner
loop
    cur_state : state := parse_stack.top.st -- peek at state at top of stack
    if cur_state = start_state and cur_sym = start_symbol
        return -- success!
    ar : action_rec := parse_tab[cur_state, cur_sym]
    case ar.action
    shift:
        parse_stack.push((cur_sym, ar.new_state))
        cur_sym := scan -- get new token from scanner
    reduce:
        cur_sym := prod_tab[ar.prod].lhs
        parse_stack.pop(prod_tab[ar.prod].rhs_len)
    shift_reduce:
        cur_sym := prod_tab[ar.prod].lhs
        parse_stack.pop(prod_tab[ar.prod].rhs_len-1)
    error:
        parse_error

```

Figure 2.28 Driver for a table-driven SLR(1) parser. We call the scanner directly, rather than using the global input.token of Figures 2.16 and 2.18, so that we can set cur_sym to be an arbitrary symbol.

39. What is the significance of the dot (•) in an LR item?
40. What distinguishes the *basis* from the *closure* of an LR state?
41. What is a *shift-reduce conflict*? How is it resolved in the various kinds of LR-family parsers?
42. Outline the steps performed by the driver of a bottom-up parser.

Parse stack	Input stream	Comment
0	read A read B ...	
0 read 1	A read B ...	shift read
0	stmt read B ...	shift id(A) & reduce by <i>stmt</i> → read id
0	stmt_list read B ...	shift <i>stmt</i> & reduce by <i>stmt_list</i> → <i>stmt</i>
0 stmt_list 2	read B sum ...	shift <i>stmt_list</i>
0 stmt_list 2 read 1	B sum := ...	shift read
0 stmt_list 2	stmt sum := ...	shift id(B) & reduce by <i>stmt</i> → read id
0	stmt_list sum := ...	shift <i>stmt</i> & reduce by <i>stmt_list</i> → <i>stmt_list stmt</i>
0 stmt_list 2	sum := A ...	shift <i>stmt_list</i>
0 stmt_list 2 id 3	:= A + ...	shift id(sum)
0 stmt_list 2 id 3 := 5	A + B ...	shift :=
0 stmt_list 2 id 3 := 5	factor + B ...	shift id(A) & reduce by <i>factor</i> → id
0 stmt_list 2 id 3 := 5	term + B ...	shift <i>factor</i> & reduce by <i>term</i> → <i>factor</i>
0 stmt_list 2 id 3 := 5 term 7	+ B write ...	shift <i>term</i>
0 stmt_list 2 id 3 := 5	expr + B write ...	reduce by <i>expr</i> → <i>term</i>
0 stmt_list 2 id 3 := 5 expr 9	+ B write ...	shift <i>expr</i>
0 stmt_list 2 id 3 := 5 expr 9	add_op B write ...	shift + & reduce by <i>add_op</i> → +
0 stmt_list 2 id 3 := 5 expr 9 add_op 10	B write sum ...	shift <i>add_op</i>
0 stmt_list 2 id 3 := 5 expr 9 add_op 10	factor write sum ...	shift id(B) & reduce by <i>factor</i> → id
0 stmt_list 2 id 3 := 5 expr 9 add_op 10	term write sum ...	shift <i>factor</i> & reduce by <i>term</i> → <i>factor</i>
0 stmt_list 2 id 3 := 5 expr 9 add_op 10 term 13	write sum ...	shift <i>term</i>
0 stmt_list 2 id 3 := 5	expr write sum ...	reduce by <i>expr</i> → <i>expr add_op term</i>
0 stmt_list 2 id 3 := 5 expr 9	write sum ...	shift <i>expr</i>
0 stmt_list 2	stmt write sum ...	reduce by <i>stmt</i> → id := <i>expr</i>
0	stmt_list write sum ...	shift <i>stmt</i> & reduce by <i>stmt_list</i> → <i>stmt</i>
0 stmt_list 2	write sum ...	shift <i>stmt_list</i>
0 stmt_list 2 write 4	sum write sum ...	shift write
0 stmt_list 2 write 4	factor write sum ...	shift id(sum) & reduce by <i>factor</i> → id
0 stmt_list 2 write 4	term write sum ...	shift <i>factor</i> & reduce by <i>term</i> → <i>factor</i>
0 stmt_list 2 write 4 term 7	write sum ...	shift <i>term</i>
0 stmt_list 2 write 4	expr write sum ...	reduce by <i>expr</i> → <i>term</i>
0 stmt_list 2 write 4 expr 6	write sum ...	shift <i>expr</i>
0 stmt_list 2	stmt write sum ...	reduce by <i>stmt</i> → write <i>expr</i>
0	stmt_list write sum ...	shift <i>stmt</i> & reduce by <i>stmt_list</i> → <i>stmt_list stmt</i>
0 stmt_list 2	write sum / ...	shift <i>stmt_list</i>
0 stmt_list 2 write 4	sum / 2 ...	shift write
0 stmt_list 2 write 4	factor / 2 ...	shift id(sum) & reduce by <i>factor</i> → id
0 stmt_list 2 write 4	term / 2 ...	shift <i>factor</i> & reduce by <i>term</i> → <i>factor</i>
0 stmt_list 2 write 4 term 7	/ 2 \$\$	shift <i>term</i>
0 stmt_list 2 write 4 term 7	mult_op 2 \$\$	shift / & reduce by <i>mult_op</i> → /
0 stmt_list 2 write 4 term 7 mult_op 11	2 \$\$	shift <i>mult_op</i>
0 stmt_list 2 write 4 term 7 mult_op 11	factor \$\$	shift number (2) & reduce by <i>factor</i> → number
0 stmt_list 2 write 4	term \$\$	shift <i>factor</i> & reduce by <i>term</i> → <i>term mult_op factor</i>
0 stmt_list 2 write 4 term 7	\$\$	shift <i>term</i>
0 stmt_list 2 write 4	expr \$\$	reduce by <i>expr</i> → <i>term</i>
0 stmt_list 2 write 4 expr 6	\$\$	shift <i>expr</i>
0 stmt_list 2	stmt \$\$	reduce by <i>stmt</i> → write <i>expr</i>
0	stmt_list \$\$	shift <i>stmt</i> & reduce by <i>stmt_list</i> → <i>stmt_list stmt</i>
0 stmt_list 2	\$\$	shift <i>stmt_list</i>
0	program	shift \$\$ & reduce by <i>program</i> → <i>stmt_list \$\$</i>
[done]		

Figure 2.29 Trace of a table-driven SLR(1) parse of the sum-and-average program. States in the parse stack are shown in boldface type. Symbols in the parse stack are for clarity only; they are not needed by the parsing algorithm. Parsing begins with the initial state of the CFSM (State 0) in the stack. It ends when we reduce by *program* → *stmt_list* \$\$, uncovering State 0 again and pushing *program* onto the input stream.

43. What kind of parser is produced by yacc/bison? By ANTLR?
44. Why are there never any epsilon productions in an LR(0) grammar?

2.3.4 Syntax Errors

EXAMPLE 2.42

A syntax error in C

Suppose we are parsing a C program and see the following code fragment in a context where a statement is expected:

```
A = B : C + D;
```

We will detect a syntax error immediately after the B, when the colon appears from the scanner. At this point the simplest thing to do is just to print an error message and halt. This naive approach is generally not acceptable, however: it would mean that every run of the compiler reveals no more than one syntax error. Since most programs, at least at first, contain numerous such errors, we really need to find as many as possible now (we'd also like to continue looking for semantic errors). To do so, we must modify the state of the parser and/or the input stream so that the upcoming token(s) are acceptable. We shall probably want to turn off code generation, disabling the back end of the compiler: since the input is not a valid program, the code will not be of use, and there's no point in spending time creating it. ■

In general, the term *syntax error recovery* is applied to any technique that allows the compiler, in the face of a syntax error, to continue looking for other errors later in the program. High-quality syntax error recovery is essential in any production-quality compiler. The better the recovery technique, the more likely the compiler will be to recognize additional errors (especially nearby errors) correctly, and the less likely it will be to become confused and announce spurious *cascading errors* later in the program.

© IN MORE DEPTH

On the PLP CD we explore several possible approaches to syntax error recovery. In *panic mode*, the compiler writer defines a small set of “safe symbols” that delimit clean points in the input. Semicolons, which typically end a statement, are a good choice in many languages. When an error occurs, the compiler deletes input tokens until it finds a safe symbol, and then “backs the parser out” (e.g., returns from recursive descent subroutines) until it finds a context in which that symbol might appear. *Phrase-level recovery* improves on this technique by employing different sets of “safe” symbols in different productions of the grammar (right parentheses when in an expression; semicolons when in a declaration). *Context-specific look-ahead* obtains additional improvements by differentiating among the various contexts in which a given production might appear in a syntax tree. To respond gracefully to certain common programming errors, the compiler writer may augment the grammar with *error productions* that capture language-specific idioms that are incorrect but are often written by mistake.

Niklaus Wirth published an elegant implementation of phrase-level and context-specific recovery for recursive descent parsers in 1976 [Wir76, Sec. 5.9]. *Exceptions* (to be discussed further in Section 8.5) provide a simpler alternative if supported by the language in which the compiler is written. For table-driven top-down parsers, Fischer, Milton, and Quiring published an algorithm in 1980 that automatically implements a well-defined notion of *locally least-cost syntax repair*. Locally least-cost repair is also possible in bottom-up parsers, but it is significantly more difficult. Most bottom-up parsers rely on more straightforward phrase-level recovery; a typical example can be found in yacc/bison.

2.4 Theoretical Foundations

Our understanding of the relative roles and computational power of scanners, parsers, regular expressions, and context-free grammars is based on the formalisms of *automata theory*. In automata theory, a *formal language* is a set of strings of symbols drawn from a finite *alphabet*. A formal language can be specified either by a set of rules (such as regular expressions or a context-free grammar) that generates the language, or by a *formal machine* that *accepts* (*recognizes*) the language. A formal machine takes strings of symbols as input and outputs either “yes” or “no.” A machine is said to accept a language if it says “yes” to all and only those strings that are in the language. Alternatively, a language can be defined as the set of strings for which a particular machine says “yes.”

Formal languages can be grouped into a series of successively larger classes known as the *Chomsky hierarchy*.¹⁴ Most of the classes can be characterized in two ways: by the types of rules that can be used to generate the set of strings, or by the type of formal machine that is capable of recognizing the language. As we have seen, *regular languages* are defined by using concatenation, alternation, and Kleene closure, and are recognized by a scanner. *Context-free languages* are a proper superset of the regular languages. They are defined by using concatenation, alternation, and recursion (which subsumes Kleene closure), and are recognized by a parser. A scanner is a concrete realization of a *finite automaton*, a type of formal machine. A parser is a concrete realization of a *push-down automaton*. Just as context-free grammars add recursion to regular expressions, push-down automata add a stack to the memory of a finite automaton. There are additional levels in the Chomsky hierarchy, but they are less directly applicable to compiler construction, and are not covered here.

It can be proven, constructively, that regular expressions and finite automata are equivalent: one can construct a finite automaton that accepts the language defined by a given regular expression, and vice versa. Similarly, it is possible to

¹⁴ Noam Chomsky (1928–), a linguist and social philosopher at the Massachusetts Institute of Technology, developed much of the early theory of formal languages.

construct a push-down automaton that accepts the language defined by a given context-free grammar, and vice versa. The grammar-to-automaton constructions are in fact performed by scanner and parser generators such as `lex` and `yacc`. Of course, a real scanner does not accept just one token; it is called in a loop so that it keeps accepting tokens repeatedly. As noted in the sidebar on page 60, this detail is accommodated by having the scanner accept the alternation of all the tokens in the language (with distinguished final states), and by having it continue to consume characters until no longer token can be constructed.

© IN MORE DEPTH

On the PLP CD we consider finite and pushdown automata in more detail. We give an algorithm to convert a DFA into an equivalent regular expression. Combined with the constructions in [Section 2.2.1](#), this algorithm demonstrates the equivalence of regular expressions and finite automata. We also consider the sets of grammars and languages that can and cannot be parsed by the various linear-time parsing algorithms.

2.5 Summary and Concluding Remarks

In this chapter we have introduced the formalisms of regular expressions and context-free grammars, and the algorithms that underlie scanning and parsing in practical compilers. We also mentioned syntax error recovery, and presented a quick overview of relevant parts of automata theory. Regular expressions and context-free grammars are language *generators*: they specify how to construct valid strings of characters or tokens. Scanners and parsers are language *recognizers*: they indicate whether a given string is valid. The principal job of the scanner is to reduce the quantity of information that must be processed by the parser, by grouping characters together into tokens, and by removing comments and white space. Scanner and parser generators automatically translate regular expressions and context-free grammars into scanners and parsers.

Practical parsers for programming languages (parsers that run in linear time) fall into two principal groups: top-down (also called LL or predictive) and bottom-up (also called LR or shift-reduce). A top-down parser constructs a parse tree starting from the root and proceeding in a left-to-right depth-first traversal. A bottom-up parser constructs a parse tree starting from the leaves, again working left-to-right, and combining partial trees together when it recognizes the children of an internal node. The stack of a top-down parser contains a prediction of what will be seen in the future; the stack of a bottom-up parser contains a record of what has been seen in the past.

Top-down parsers tend to be simple, both in the parsing of valid strings and in the recovery from errors in invalid strings. Bottom-up parsers are more powerful, and in some cases lend themselves to more intuitively structured grammars,

though they suffer from the inability to embed action routines at arbitrary points in a right-hand side (we discuss this point in more detail in Section 4.5.1). Both varieties of parser are used in real compilers, though bottom-up parsers are more common. Top-down parsers tend to be smaller in terms of code and data size, but modern machines provide ample memory for either.

Both scanners and parsers can be built by hand if an automatic tool is not available. Handbuilt scanners are simple enough to be relatively common. Handbuilt parsers are generally limited to top-down recursive descent, and are most commonly used for comparatively simple languages (e.g., Pascal but not Ada). Automatic generation of the scanner and parser has the advantage of increased reliability, reduced development time, and easy modification and enhancement.

Various features of language design can have a major impact on the complexity of syntax analysis. In many cases, features that make it difficult for a compiler to scan or parse also make it difficult for a human being to write correct, maintainable code. Examples include the lexical structure of Fortran and the `if... then... else` statement of languages like Pascal. This interplay among language design, implementation, and use will be a recurring theme throughout the remainder of the book.

2.6 Exercises

2.1 Write regular expressions to capture the following.

- (a) Strings in C. These are delimited by double quotes ("), and may not contain newline characters. They may contain double-quote or backslash characters if and only if those characters are “escaped” by a preceding backslash. You may find it helpful to introduce shorthand notation to represent any character that is *not* a member of a small specified set.
- (b) Comments in Pascal. These are delimited by (* and *) or by { and }.
- (c) Numeric constants in C. These are octal, decimal, or hexadecimal integers, or decimal or hexadecimal floating-point values. An octal integer begins with 0, and may contain only the digits 0–7. A hexadecimal integer begins with 0x or 0X, and may contain the digits 0–9 and a/A–f/F. A decimal floating-point value has a fractional portion (beginning with a dot) or an exponent (beginning with E or e). Unlike a decimal integer, it is allowed to start with 0. A hexadecimal floating-point value has an optional fractional portion and a mandatory exponent (beginning with P or p). In either decimal or hexadecimal, there may be digits to the left of the dot, the right of the dot, or both, and the exponent itself is given in decimal, with an optional leading + or – sign. An integer may end with an optional U or u (indicating “unsigned”), and/or L or l (indicating “long”) or LL or ll (indicating “long long”). A floating-point value may

end with an optional F or f (indicating “float”—single precision) or L or l (indicating “long”—double precision).

- (d) Floating-point constants in Ada. These match the definition of *real* in [Example 2.3](#) [page 44]), except that (1) a digit is required on both sides of the decimal point, (2) an underscore is permitted between digits, and (3) an alternative numeric base may be specified by surrounding the nonexponent part of the number with pound signs, preceded by a base in decimal (e.g., 16#6.a7#e+2). In this latter case, the letters a..f (both upper- and lowercase) are permitted as digits. Use of these letters in an inappropriate (e.g., decimal) number is an error, but need not be caught by the scanner.
 - (e) Inexact constants in Scheme. Scheme allows real numbers to be explicitly *inexact* (imprecise). A programmer who wants to express all constants using the same number of characters can use sharp signs (#) in place of any lower-significance digits whose values are not known. A base-10 constant without exponent consists of one or more digits followed by zero or more sharp signs. An optional decimal point can be placed at the beginning, the end, or anywhere in-between. (For the record, numbers in Scheme are actually a good bit more complicated than this. For the purposes of this exercise, please ignore anything you may know about sign, exponent, radix, exactness and length specifiers, and complex or rational values.)
 - (f) Financial quantities in American notation. These have a leading dollar sign (\$), an optional string of asterisks (*—used on checks to discourage fraud), a string of decimal digits, and an optional fractional part consisting of a decimal point (.) and two decimal digits. The string of digits to the left of the decimal point may consist of a single zero (0). Otherwise it must not start with a zero. If there are more than three digits to the left of the decimal point, groups of three (counting from the right) must be separated by commas (,). Example: \$**2,345.67. (Feel free to use “productions” to define abbreviations, so long as the language remains regular.)
- 2.2 Show (as “circles-and-arrows” diagrams) the finite automata for [Exercise 2.1](#).
- 2.3 Build a regular expression that captures all nonempty sequences of letters other than `file`, `for`, and `from`. For notational convenience, you may assume the existence of a **not** operator that takes a set of letters as argument and matches any *other* letter. Comment on the practicality of constructing a regular expression for all sequences of letters other than the keywords of a large programming language.
- 2.4 (a) Show the NFA that results from applying the construction of [Figure 2.7](#) to the regular expression `letter ( letter | digit )*`.
- (b) Apply the transformation illustrated by [Example 2.14](#) to create an equivalent DFA.

- (c) Apply the transformation illustrated by [Example 2.15](#) to minimize the DFA.
- 2.5 Starting with the regular expressions for *integer* and *decimal* in [Example 2.3](#), construct an equivalent NFA, the set-of-subsets DFA, and the minimal equivalent DFA. Be sure to keep separate the final states for the two different kinds of token (see the sidebar on page 60). You will find the exercise easier if you undertake it by modifying the machines in [Examples 2.13 through 2.15](#).
- 2.6 Build an ad hoc scanner for the calculator language. As output, have it print a list, in order, of the input tokens. For simplicity, feel free to simply halt in the event of a lexical error.
- 2.7 Write a program in your favorite scripting language to remove comments from programs in the calculator language ([Example 2.9](#)).
- 2.8 Build a nested-case-statements finite automaton that converts all letters in its input to lowercase, except within Pascal-style comments and strings. A Pascal comment is delimited by { and }, or by (* and *). Comments do not nest. A Pascal string is delimited by single quotes (' ... '). A quote character can be placed in a string by doubling it ('Madam, I 'm Adam.'). This upper-to-lower mapping can be useful if feeding a program written in standard Pascal (which ignores case) to a compiler that considers upper- and lowercase letters to be distinct.
- 2.9 (a) Describe in English the language defined by the regular expression $a^*(b a^* b a^*)^*$. Your description should be a high-level characterization—one that would still make sense if we were using a different regular expression for the same language.
- (b) Write an unambiguous context-free grammar that generates the same language.
- (c) Using your grammar from part (b), give a canonical (rightmost) derivation of the string $b a a b a a b b$.
- 2.10 Give an example of a grammar that captures right associativity for an exponentiation operator (e.g., ** in Fortran).
- 2.11 Prove that the following grammar is LL(1):

$$\begin{aligned} \text{decl} &\longrightarrow \text{ID } \text{decl_tail} \\ \text{decl_tail} &\longrightarrow , \text{ decl} \\ &\longrightarrow : \text{ ID } ; \end{aligned}$$

(The final ID is meant to be a type name.)

- 2.12 Consider the following grammar:

$$\begin{aligned} G &\longrightarrow S \$\$ \\ S &\longrightarrow A M \\ M &\longrightarrow S \mid \epsilon \end{aligned}$$

$$\begin{aligned}
 A &\longrightarrow a E \mid b A A \\
 E &\longrightarrow a B \mid b A \mid \epsilon \\
 B &\longrightarrow b E \mid a B B
 \end{aligned}$$

- (a) Describe in English the language that the grammar generates.
- (b) Show a parse tree for the string $a b a a$.
- (c) Is the grammar LL(1)? If so, show the parse table; if not, identify a prediction conflict.

2.13 Consider the following grammar:

$$\begin{aligned}
 stmt &\longrightarrow assignment \\
 &\quad \longrightarrow subr_call \\
 assignment &\longrightarrow id := expr \\
 subr_call &\longrightarrow id (arg_list) \\
 expr &\longrightarrow primary expr_tail \\
 expr_tail &\longrightarrow op expr \\
 &\quad \longrightarrow \epsilon \\
 primary &\longrightarrow id \\
 &\quad \longrightarrow subr_call \\
 &\quad \longrightarrow (expr) \\
 op &\longrightarrow + \mid - \mid * \mid / \\
 arg_list &\longrightarrow expr args_tail \\
 args_tail &\longrightarrow , arg_list \\
 &\quad \longrightarrow \epsilon
 \end{aligned}$$

- (a) Construct a parse tree for the input string $foo(a, b)$.
- (b) Give a canonical (rightmost) derivation of this same string.
- (c) Prove that the grammar is not LL(1).
- (d) Modify the grammar so that it is LL(1).

2.14 Consider the language consisting of all strings of properly balanced parentheses and brackets.

- (a) Give LL(1) and SLR(1) grammars for this language.
- (b) Give the corresponding LL(1) and SLR(1) parsing tables.
- (c) For each grammar, show the parse tree for $([] ([])) [] (())$.
- (d) Give a trace of the actions of the parsers in constructing these trees.

2.15 Consider the following context-free grammar:

$$\begin{aligned}
 G &\longrightarrow G B \\
 &\quad \longrightarrow G N \\
 &\quad \longrightarrow \epsilon
 \end{aligned}$$

$$\begin{aligned}
B &\longrightarrow (E) \\
E &\longrightarrow E (E) \\
&\longrightarrow \epsilon \\
N &\longrightarrow (L] \\
L &\longrightarrow L E \\
&\longrightarrow L (\\
&\longrightarrow \epsilon
\end{aligned}$$

- (a) Describe, in English, the language generated by this grammar. (Hint: B stands for “balanced”; N stands for “nonbalanced”.) (Your description should be a high-level characterization of the language—one that is independent of the particular grammar chosen.)
 - (b) Give a parse tree for the string $((] ()$.
 - (c) Give a canonical (rightmost) derivation of this same string.
 - (d) What is $\text{FIRST}(E)$ in our grammar? What is $\text{FOLLOW}(E)$? (Recall that FIRST and FOLLOW sets are defined for symbols in an arbitrary CFG, regardless of parsing algorithm.)
 - (e) Given its use of left recursion, our grammar is clearly not LL(1). Does this language have an LL(1) grammar? Explain.
- 2.16 Give a grammar that captures all levels of precedence for arithmetic expressions in C, as shown in [Figure 6.1](#) (page 223). (Hint: This exercise is somewhat tedious. You’ll probably want to attack it with a text editor rather than a pencil.)
- 2.17 Extend the grammar of [Figure 2.24](#) to include `if` statements and `while` loops, along the lines suggested by the following examples:

```

abs := n
if n < 0 then abs := 0 - abs fi

sum := 0
read count
while count > 0 do
    read n
    sum := sum + n
    count := count - 1
od
write sum

```

Your grammar should support the six standard comparison operations in conditions, with arbitrary expressions as operands. It also should allow an arbitrary number of statements in the body of an `if` or `while` statement.

- 2.18 Consider the following LL(1) grammar for a simplified subset of Lisp:

$$\begin{aligned}
P &\longrightarrow E \$\$ \\
E &\longrightarrow \text{atom} \\
&\longrightarrow ' E \\
&\longrightarrow (E Es) \\
Es &\longrightarrow E Es \\
&\longrightarrow
\end{aligned}$$

- (a) What is $\text{FIRST}(Es)$? $\text{FOLLOW}(E)$? $\text{PREDICT}(Es \longrightarrow \epsilon)$?
 - (b) Give a parse tree for the string $(\text{cdr } '(\text{a b c})) \$\$$.
 - (c) Show the leftmost derivation of $(\text{cdr } '(\text{a b c})) \$\$$.
 - (d) Show a trace, in the style of [Figure 2.20](#), of a table-driven top-down parse of this same input.
 - (e) Now consider a recursive descent parser running on the same input. At the point where the quote token $(')$ is matched, which recursive descent routines will be active (i.e., what routines will have a frame on the parser's run-time stack)?
- 2.19 Write top-down and bottom-up grammars for the language consisting of all well-formed regular expressions. Arrange for all operators to be left-associative. Give Kleene closure the highest precedence and alternation the lowest precedence.
 - 2.20 Suppose that the expression grammar in [Example 2.8](#) were to be used in conjunction with a scanner that did *not* remove comments from the input, but rather returned them as tokens. How would the grammar need to be modified to allow comments to appear at arbitrary places in the input?
 - 2.21 Build a complete recursive descent parser for the calculator language. As output, have it print a trace of its matches and predictions.
 - 2.22 Flesh out the details of an algorithm to eliminate left recursion and common prefixes in an arbitrary context-free grammar.
 - 2.23 In some languages an assignment can appear in any context in which an expression is expected: the value of the expression is the right-hand side of the assignment, which is placed into the left-hand-side as a side effect. Consider the following grammar fragment for such a language. Explain why it is not $\text{LL}(1)$, and discuss what might be done to make it so.

$$\begin{aligned}
\text{expr} &\longrightarrow \text{id} := \text{expr} \\
&\longrightarrow \text{term } \text{term_tail} \\
\text{term_tail} &\longrightarrow + \text{term } \text{term_tail} \mid \epsilon \\
\text{term} &\longrightarrow \text{factor } \text{factor_tail} \\
\text{factor_tail} &\longrightarrow * \text{factor } \text{factor_tail} \mid \epsilon \\
\text{factor} &\longrightarrow (\text{expr}) \mid \text{id}
\end{aligned}$$

- 2.24 Construct the CFSM for the *id.list* grammar in [Example 2.20](#) (page 67) and verify that it can be parsed bottom-up with *zero* tokens of look-ahead.

- 2.25 Modify the grammar in [Exercise 2.24](#) to allow an *id_list* to be empty. Is the grammar still LR(0)?
- 2.26 Consider the following grammar for a declaration list:

$$\begin{aligned} \text{decl_list} &\rightarrow \text{decl_list decl ;} \mid \text{decl ;} \\ \text{decl} &\rightarrow \text{id : type} \\ \text{type} &\rightarrow \text{int} \mid \text{real} \mid \text{char} \\ &\rightarrow \text{array const .. const of type} \\ &\rightarrow \text{record decl_list end} \end{aligned}$$

Construct the CFSM for this grammar. Use it to trace out a parse (as in [Figure 2.29](#)) for the following input program:

```
foo : record
    a : char;
    b : array 1 .. 2 of real;
end;
```

- 2.27 The dangling else problem of Pascal is not shared by Algol 60. To avoid ambiguity regarding which then is matched by an else, Algol 60 prohibits if statements immediately inside a then clause. The Pascal fragment

```
if C1 then if C2 then S1 else S2
```

must be written as either

```
if C1 then begin if C2 then S1 end else S2
```

or

```
if C1 then begin if C2 then S1 else S2 end
```

in Algol 60. Show how to write a grammar for conditional statements that enforces this rule. (Hint: you will want to distinguish in your grammar between conditional statements and nonconditional statements; some contexts will accept either, some only the latter.)

- © 2.28–2.32 In More Depth.

2.7 Explorations

- 2.33 Some languages (e.g., C) distinguish between upper- and lowercase letters in identifiers. Others (e.g., Ada) do not. Which convention do you prefer? Why?

- 2.34 The syntax for type casts in C and its descendants introduces potential ambiguity: is $(x)-y$ a subtraction, or the unary negation of y , cast to type x ? Find out how C, C++, Java, and C# answer this question. Discuss how you would implement the answer(s).
- 2.35 What do you think of Haskell, Occam, and Python's use of indentation to delimit control constructs ([Section 2.1.1](#))? Would you expect this convention to make program construction and maintenance easier or harder? Why?
- 2.36 Skip ahead to [Section 13.4.2](#) and learn about the “regular expressions” used in scripting languages, editors, search tools, and so on. Are these really regular? What can they express that cannot be expressed in the notation introduced in [Section 2.1.1](#)?
- 2.37 Rebuild the automaton of [Exercise 2.8](#) using `lex/flex`.
- 2.38 Find a manual for `yacc/bison`, or consult a compiler textbook [[ALSU07](#), Secs. 4.8.1 and 4.9.2] to learn about *operator precedence parsing*. Explain how it could be used to simplify the grammar of [Exercise 2.16](#).
- 2.39 Use `lex/flex` and `yacc/bison` to construct a parser for the calculator language. Have it output a trace of its shifts and reductions.
- 2.40 Repeat the previous exercise using ANTLR.
- © 2.41–2.42 In More Depth.

2.8 Bibliographic Notes

Our coverage of scanning and parsing in this chapter has of necessity been brief. Considerably more detail can be found in texts on parsing theory [[AU72](#)] and compiler construction [[ALSU07](#), [FL88](#), [App97](#), [GBJL01](#), [CT04](#)]. Many compilers of the early 1960s employed recursive descent parsers. Lewis and Stearns [[LS68](#)] and Rosenkrantz and Stearns [[RS70](#)] published early formal studies of LL grammars and parsing. The original formulation of LR parsing is due to Knuth [[Knu65](#)]. Bottom-up parsing became practical with DeRemer's discovery of the SLR and LALR algorithms [[DeR71](#)]. W. L. Johnson et al. [[JPAP68](#)] describe an early scanner generator. The Unix `lex` tool is due to Lesk [[Les75](#)]. `Yacc` is due to S. C. Johnson [[Joh75](#)].

Further details on formal language theory can be found in a variety of textbooks, including those of Hopcroft, Motwani, and Ullman [[HMU01](#)] and Sipser [[Sip97](#)]. Kleene [[Kle56](#)] and Rabin and Scott [[RS59](#)] proved the equivalence of regular expressions and finite automata.¹⁵ The proof that finite automata are unable to

15 Dana Scott (1932–), Professor Emeritus at Carnegie Mellon University, is known principally for inventing domain theory and launching the field of denotational semantics, which provides a mathematically rigorous way to formalize the meaning of programming languages. Michael Rabin (1931–), of Harvard University, has made seminal contributions to the concepts of non-determinism and randomization in computer science. Scott and Rabin shared the ACM Turing Award in 1976.

recognize nested constructs is based on a theorem known as the *pumping lemma*, due to Bar-Hillel, Perles, and Shamir [BHPS61]. Context-free grammars were first explored by Chomsky [Cho56] in the context of natural language. Independently, Backus and Naur developed BNF for the syntactic description of Algol 60 [NBB⁺63]. Ginsburg and Rice [GR62] recognized the equivalence of the two notations. Chomsky [Cho62] and Evey [Eve63] demonstrated the equivalence of context-free grammars and push-down automata.

Fischer and LeBlanc's text [FL88] contains an excellent survey of error recovery and repair techniques, with references to other work. The phrase-level recovery mechanism for recursive descent parsers described in Section ©2.3.4 is due to Wirth [Wir76, Sec. 5.9]. The locally least-cost recovery mechanism for table-driven LL parsers described in Section ©2.3.4 is due to Fischer, Milton, and Quiring [FMQ80]. Dion published a locally least-cost bottom-up repair algorithm in 1978 [Dio78]. It is quite complex, and requires very large precomputed tables. McKenzie, Yeatman, and De Vere subsequently showed how to effect the same repairs without the precomputed tables, at a higher but still acceptable cost in time [MYD95].